

SuperLocalMemory 4.0: The Governed Memory Operating System for AI Agents

Unifying Information-Geometric Retrieval, Bi-Temporal Recall,
Verifiable Transactions, and Multi-Tenant Governance in One Local-First Control Plane
—and Two Invariants That Ask Whether a Wired Mechanism Takes Effect

Varun Pratap Bhardwaj¹ Garima Singh² Arun Pratap Bhardwaj²

¹Qualixar / Independent Researcher, India ²Independent Researcher, India

varun.pratap.bhardwaj@gmail.com

garima1213@gmail.com arun.pratap.bhardwaj@gmail.com

ORCID (Varun Pratap Bhardwaj): 0009-0002-8726-4289

Abstract

In plain terms. SuperLocalMemory is an open, local-first memory system for AI agents. It lets agents remember across sessions, machines, and people; it can run entirely on hardware you control, with no cloud provider in the memory path; and a team or a company can operate it without giving up isolation between one person’s context and another’s. What distinguishes it is less the feature list than the evidence standard applied to it: every reliability guarantee is stated as a falsifiable invariant, tested under an adversarial condition, bracketed so that it fails if the mechanism does nothing, positive-controlled where the design admits one, and reported together with an explicit account of what the experiment did *not* exercise. We call that practice **AI Reliability Engineering**, and this paper is an attempt to hold a memory system to it rather than to report capability numbers alone.

This version applies that standard to our own published claims and retracts two of them—a learning layer we called governed, and a governance-cost figure that measured something other than its label. Retracting a number is the standard working, not the standard failing. A governed write costs 17.2ms at the median on a freshly created store.

The finding this version leads with. Ten mechanisms in this system were built correctly, wired into a live call path, and left inert or unobservable at their final connection—among them a monitor inferring liveness from the output it gates, an availability check that refuses a working store, and a published latency figure that subtracted two write paths which were never built to be compared, so the number it produced was never an overhead. Each was silent: nothing raised, nothing logged, every file present and every version correct. The common shape is that *implemented*, *reachable* and *effective* are three different questions, and that a signal derived from the mechanism it evaluates cannot detect that mechanism’s failure. We contribute two mechanical invariants—one asserting that a Bayesian posterior’s mean has moved off its prior value after n observations, one asserting whether a schema-guarded path can execute against this store at all, and naming where its missing data actually lives when it cannot—that answer the third question directly. Pointed at our own store they returned STALLED on every arm of two learners, and acting on that verdict named the cause: two individually correct components disagreed on an identifier namespace, so no engagement signal could ever reach the learner, while a neutral-by-default settlement shrank posterior variance and made the learner steadily harder to move. **Neutral is a commitment, not an abstention.** The invariants ship in 4.1.5 and the repairs in 4.1.6–4.1.9. We then test the repair rather than assert it: a three-arm ablation varying only the identifier namespace, with

every production module in the path held identical, moves **no posterior in 120 recalls** with the defect present and **every instantiated arm** with it absent, while a negative control that writes every outcome ticket but supplies no engagement correctly settles nothing. The loop closes, and we are explicit that a controlled ablation is not a deployment result.

The remainder of this abstract is technical.

AI agents are moving from single-user assistants to shared infrastructure—teams, companies, and fleets of autonomous workers that must remember across sessions, machines, and people without leaking one context into another, and, for regulated or data-resident organizations, without a cloud provider in the memory path. Existing work answers this in *fragments*—memory SDKs and managed platforms, temporal context graphs, stateful agent runtimes, framework-native primitives, context APIs, and user-timeline stores—each strong on one axis. Few open, local-first runtimes unify retrieval quality, a learning brain, time-awareness, a knowledge graph, verifiable governance, and multi-tenant isolation in a single system an organization can operate.

We present **SuperLocalMemory 4.0 (SLM 4.0)**, a governed, local-first *memory operating system* for AI agents: one control plane, organized as seven operating layers over six managed SQLite stores plus vector and graph projections, reachable through a CLI, an MCP server, an HTTP daemon, a dashboard, an editor plugin, and separately packaged adapters for nine agent frameworks, in three operating modes (fully local; local with an on-device model; provider-assisted). SLM 4.0 unifies, in one runtime: (i) **multi-channel retrieval**—dense semantic, BM25 lexical, temporal, Hopfield-associative, and spreading-activation candidates fused by reciprocal-rank fusion, over an information-geometric scoring and lifecycle substrate; (ii) a **learning and behavior layer that is instrumented, measured, and reported inert**—feedback capture, reward routing, a Thompson-sampling channel selector, a learning-to-rank stage, behavioral soft-prompt injection, and a blind-verified, budgeted skill-evolution pipeline that bounds self-modification. **We do not claim this layer learns; we measured that it does not.** Across three independent stores its Thompson posteriors satisfy $\alpha - \alpha_0 = \beta - \beta_0 = n/2$ *exactly* on **459 of 459 arms** after **5,657** plays; a second Beta learner shows the identical signature; and trust-weighted forgetting, which our prior work reported as delivered, is called for every fact on the hot path and is *arithmetically inert*, its guard testing for a column absent from the table it tests while that column sits populated on 4,340 rows of a neighbouring one (Section 7); (iii) **SLM-Mesh**, serverless per-tenant-isolated coordination through a local SQLite broker—authenticated peer messages, shared state, and advisory locks—extended (opt-in) across machines with deterministic convergence and fencing-token lock ordering (*scope*: coordination, *not* linearizable consensus, quorum, or a conflict-free replicated database); (iv) **time-aware memory**—a three-date bi-temporal model, a dedicated temporal retrieval channel, superseded-fact demotion, an Ebbinghaus recency model, and cross-surface **as_of** recall; and (v) **governance-native memory**—multi-scope tenant isolation, role-based access, a login gate, GDPR export/erasure with tamper-evident receipts, a hash-chained audit trail, a deployment-context EU AI Act checklist, and *operable recovery*—all in a local-first runtime. These capabilities become trustworthy under one governing design goal—one authenticated actor, one profile generation, one policy decision, one durable receipt, one verifiable completion state—implemented by four reliability mechanisms: generation-fenced write admission, verifiable memory transactions (a transactional obligation ledger with per-projection apply/verify/compensate/erase owners and a hash-checkable completion manifest), cross-store verified erasure over those owners, and bi-temporal **as_of** demotion. V4 enforces the invariant on the primary write path; other transports adopt the same gateway incrementally, so we present it as the enforced design of that path and the target architecture for the platform, not as a property already enforced on every surface.

We evaluate the reliability spine by direct measurement against the candidate artifact: eleven fault-injection and mechanism scenarios, each repeated 200 times as a deterministic flakiness check. **2,199 of 2,200 repetitions upheld their scoped component property**, and the one that did not is reported rather than re-run: the generation-fence experiment held 199 of 200 because the writer was transiently unavailable, not because the fence misjudged an epoch, and diagnosing it exposed one more instance of the same failure class (these repetitions are not independent Bernoulli trials and support no population failure-rate inference; no

single experiment exercises the full end-to-end path—transport, multi-process, mesh, and long-lived deployment fault-injection are future work). We report real-scale performance on a 1,232 MiB memory-store copy. **We withdraw the governed write-envelope *overhead* figure from the previous version.** The two arms differ in ways other than governance, and this version withdraws *two* separate accounts of which way mattered—both checkable, both wrong. The difference is negative at the median, which no genuine overhead can be. We report the three paths as absolute costs and then answer the question properly by abandoning subtraction: timing the envelope’s components where they run gives a governed write of 10.43 ms p50 of which the envelope is 70.3%—and **almost all of that is durability, not governance.** The generation fence costs 1.9 μ s and the obligation ledger 42 μ s, together four tenths of one per cent; the rest is an encrypted write-ahead journal. We carry the published LoCoMo evidence unchanged, at its stated coverage (protocol-scoped, *not* a new V4 benchmark). Several of these capabilities exist individually in prior and commercial art; our contribution is their *integration* into one local-first, verifiable, governed runtime, and the central systems contribution is a hash-verifiable projection-obligation and completion-manifest protocol across heterogeneous local stores under scoped admission.

1 Introduction

The last two years produced a rich literature on agent memory. Systems have advanced extraction and consolidation [1], temporal knowledge graphs [22], neurobiologically inspired retrieval [5, 6], OS-style hierarchical tiers [9, 18], memory as an OS-managed resource [12], multi-type and multi-agent coordination [25], efficient tiering with offline consolidation [3], strategic forgetting [27], and trust-gated retrieval [31]. Each improves one dimension of memory quality.

From point solutions to a complete memory layer. A team standing up durable agent memory today still assembles it from specialized parts—each with its own trust boundary, failure model, and audit story. What we did not find, in a dated review of representative systems (Section 2), is a single runtime in which retrieval quality, a learning brain, time-awareness, a knowledge graph, verifiable transactions, and multi-tenant governance are the *same* system, running on infrastructure the organization controls. This paper contributes that runtime.

Rent the LLM, own the memory. Foundation models are converging into an interchangeable, rented capability; what an organization actually accumulates and must protect is its *memory*—the context, decisions, entities, skills, and provenance its agents build over time. That asset, and the governance over it, should be owned and operated by the organization on hardware it controls. A complete memory layer is therefore local-first *by construction*: storage, retrieval, the learning brain, governance, and optimization all run on the organization’s own infrastructure, with no provider required in the memory path (Modes A and B; Mode C is provider-assisted).

The operating questions. A second class of questions becomes central the moment agent memory becomes shared organizational infrastructure: Can one client’s memory surface in another’s recall—on *any* retrieval path? When a user exercises the right to be forgotten, does the fact disappear from every registered projection store, with evidence? When agents on several laptops collaborate, can they share what they should without a cloud server? When an agent improves its own skills, can the organization bound and audit that self-modification? These are the operating conditions of team-scale, multi-surface agent memory. They are the province of **AI Reliability Engineering**—the systematic application of reliability, correctness, and governance principles to AI-augmented software—and SLM 4.0 is built for them.

The governing invariant. SLM 4.0’s governing design goal is a single admission invariant—one authenticated actor, one profile generation, one policy decision, one durable operation receipt, and one verifiable completion state across the registered projection owners. V4 realises this invariant on the primary write path (the HTTP `/remember` route and internal ingestion admission), where the operation policy registry is evaluated and the generation fence applies; the other transports (MCP, CLI, WebSocket) and operation kinds adopt the same gateway

incrementally. We present the invariant as the enforced design of the write path and the target architecture for the platform—not as a property already enforced on every surface.

Contributions. SLM 4.0 consolidates a line of work—trust and behavioral foundations [19], an information-geometric retrieval substrate [21], and a living-brain learning and lifecycle model [20]—into one production system. On those foundations, V4 contributes and *measures* the reliability spine: generation-fenced admission, verifiable memory transactions with a hash-checkable completion manifest, cross-store verified erasure, and a cross-surface operation policy registry. Section 13 states each contribution (C1–C8) with its defensible claim and explicit scope limit.

2 Related Work

We organize related work by the capability each system most advances and locate SLM 4.0’s distinct contribution in each area.

Extraction and retrieval quality. Mem0 [1, 17] pairs LLM-based extraction with consolidation and reports strong LoCoMo results; its 2026 revision adds cross-memory entity linking and temporal accuracy gains [16]. HippoRAG [5, 6] models hippocampal indexing with Personalized PageRank. Many retrieval-quality systems treat tenant isolation as application metadata rather than a write-time fenced invariant; in the corpus we reviewed, we did not find one that specifies per-operation transactional consistency across multiple heterogeneous derived stores. SLM 4.0’s hybrid retrieval applies established hybrid-retrieval principles; we do not claim retrieval-architecture novelty.

Temporal and knowledge-graph memory. Graphiti/Zep [22, 30] maintains bi-temporal edge validity with automatic invalidation—temporal correctness at the *storage* level. SLM 4.0 places time at the *retrieval-fusion* level: a dedicated temporal candidate channel, non-destructive demotion of superseded facts, and an Ebbinghaus-parameterized recency model applied per fact after RRF fusion. We do not claim end-to-end temporal accuracy superiority; Graphiti’s bi-temporal storage is established prior art, and recent extraction-level results [16] raise that bar.

OS-inspired memory. MemGPT/Letta [10, 18], MemoryOS [9], and MemOS [12] apply the OS metaphor to hierarchical context tiers and MemCube scheduling. We reuse the OS metaphor but apply it to authorization, multi-store durability, and governance. We do not claim the OS framing itself as novel.

Self-improvement and governed skill evolution. The memory literature addresses improvement at the level of memory *content*—profile personalization, note re-linking, and sleep-time consolidation. We are not aware of a prior system (in the corpus reviewed through 2026-08-03) that governs the mutation of live agent *skill* configurations under a staged **screen** → **confirm** → **mutate** **blind-verify** → **persist** pipeline with candidate provenance, budget controls, and hash-linked state-transition history.

Multi-tenancy, governance, and local-first. Multi-tenant isolation, compliance controls, and local-first operation are well represented—Supermemory [23], Mem0 [17], and Zep/Graphiti [30] all provide strong posture on these axes individually. Governed multi-tenant shared memory is itself concurrent work: MemClaw [15] provides scoped retrieval, temporal supersession, and policy-governed propagation; a production governed-memory architecture has been described [24]. SLM 4.0’s contribution is the *integration* into a single local-first runtime: a scope algebra enforced by profile-scoped predicates across all retrieval channels, a write-time generation fence, and erasure verified across the three registered projection owners—with isolation and projection erasure *measured*.

Concurrent transactional memory work. Two July-2026 preprints address transactional agent memory at distinct layers. MemTX [11] stages writes in snapshot-isolated belief transactions with cascading repair—correctness at the **semantic/belief layer**. MemTxn [2] defines a transaction boundary for source-supported updates with complete-state recovery—correctness at

the **logical-state layer**. SLM 4.0’s transactions center on a **distinct layer: physical-store projection consistency**—were all registered projection owners updated and independently verified, under one hash-checkable completion manifest with an honest DEGRADED state? These layers compose: SLM 4.0’s manifest could sit beneath a MemTX belief transaction. We claim no priority over this concurrent work.

Self-organizing memory operating systems. EverMemOS [32] presents a *self-organizing memory operating system*: dialogue is converted into memory cells carrying episodic traces, atomic facts and time-bounded foresight signals, consolidated into thematic scenes, and retrieved by scene-guided agentic recollection. The two systems self-organize different things. EverMemOS self-organizes *content*: it decides what to keep, how to group it, and what to reconstruct, and it is evaluated on reasoning quality. SLM 4.0 governs *operations*: which writes are admitted, whether every projection of an admitted write was applied and verified, what may be erased and provably was, and—in this version—what is permitted to change the agent’s behaviour. A system could adopt both, and the comparison is not competitive: we report no retrieval-quality result for V4. It shares vocabulary with this work—atomic facts, foresight, scenes—and the operating-system framing that MemOS [12] established before either of us. MindMemOS [35] and AgentMemBench [36] appeared after the first version of this paper; the former on portable self-evolving memory layers, the latter benchmarking memory-management strategies.

Governed memory as a framework. Two lines of work use the term. A production governed-memory architecture for multi-agent workflows has been described [24], and SSGM [34] proposes a conceptual governance framework for evolving agent memory built on consistency verification, temporal decay modelling, and access control applied before consolidation. Both are close in intent to this work, and SSGM’s decay and pre-consolidation-control mechanisms are conceptually adjacent to the trust-modulated retention and write admission described here. The difference we claim is not the concept but its state: the mechanisms in this paper are implemented, exercised, and—where they do not work—*measured and reported as not working* (Section 7).

Diagnosing memory failure. MemFail [33] is the nearest prior art to Section 11.4. It formalises a memory system as the composition of summarization, storage and retrieval, derives the failure modes each operation admits, and builds five adversarially designed datasets to attribute a wrong answer to a specific operation across four systems treated as black boxes. It is a diagnostic benchmark applied from outside, and it is the right instrument for the question it asks.

Two further works bound the same space and must be named. Wu [37] is the closest of all: a longitudinal taxonomy of silent failures in a production LLM agent runtime, eight weeks of incidents with postmortems, five mechanism classes, and the finding that most were caught by human observation rather than by tests or audits. Its organising pattern is *an error signal that never reaches a human in actionable form*—an error occurs and is swallowed, diluted, or, in its most striking class, narrated into fluent prose by the model itself. Mao et al. [39] attacks the same word from the opposite end, tracing feature circuits inside the model to localise a silent memory failure to the responsible stage.

The systems lineage we sit in. Before positioning our class we should say plainly where the idea comes from, because it is not ours. Saltzer et al. [40] established forty years ago that a guarantee cannot be established by a lower layer alone; it has to be checked at the level where it is supposed to hold. Huang et al. [41] named *differential observability*: a system’s own failure detectors report health while the clients that depend on it observe failure, which is the same asymmetry we describe when a learner’s counters rise while its posterior means do not move. Huang et al. [42] answered it by turning callers into observers of the components they call—an independent observer, placed outside the failing component—and detected every one of fifteen reproduced gray failures. Lou et al. [43] pushed further into our territory, inferring semantic

rules from a system’s own regression tests and enforcing them at runtime across ZooKeeper, HDFS and Kafka to catch violations that raise nothing; Souza et al. [44] continues that line with learned checker inference.

So the principle is old and well developed: **effectiveness is an end-to-end semantic property, and establishing it requires an oracle that is independent of the mechanism under test.** We claim no part of that. What we contribute is its application to a place this literature has not looked—the learning and schema-guarded paths of an agent memory system—and two concrete checkers for that setting. It is also worth saying what is different about our setting: the distributed-systems work above detects failures that some client eventually experiences, whereas a memory system’s learner can be inert for months with no client anywhere observing anything wrong, because a uniformly-selecting retriever still returns plausible memories. There is no downstream observer to recruit. The oracle has to be constructed from the mechanism’s own persisted state, and constructed so that it does not depend on the mechanism’s own account of itself.

Our class sits between the two agent-memory works above and overlaps neither. **In our silent instances there is no error event at all.** A Thompson posterior receiving a valid neutral reward forever has nothing to swallow; a schema guard returning `False` is behaving exactly as designed; a latency delta between two arms that were never comparable is a correct subtraction of the wrong quantities. There is no exception to lose, no log line suppressed, and no stage to localise—the computation runs, returns, and means nothing. Where Wu [37] asks why a failure’s signal did not reach a human, we ask why there was no signal to begin with, and where Mao et al. [39] instruments the model, we query the memory system’s own persisted state. The three are complementary, and a deployment would want all three.

Our failures are equally not on MemFail’s axis, which is why we cite it as a boundary rather than as a baseline. None of summarization, storage or retrieval is where a learner that recorded five thousand plays and moved no posterior fails; nor a conditional path that never executed because its guard reads a neighbouring table’s column; nor a liveness probe that infers health from the output it gates. These are failures of *wiring*, not of answer quality, and they are invisible to a black-box benchmark because the system returns a plausible answer throughout. Attributing them requires operating the system and querying its stores, which is what Section 11.4 reports.

The second of our two invariants has a lineage we should name. Asserting that a conditional path has actually executed in production is well established in software engineering as dead-conditional and flag-controlled dead-code analysis, where a branch guarded by a long-pinned flag is syntactically reachable, appears in coverage, and has been dead for months. We claim no novelty for that idea. What we add is narrow: applying it to *schema-presence* guards in a memory system’s learning path, where the flag is the existence of a database column, the consequence is a Bayesian learner that is arithmetically inert rather than merely switched off, and the check reports where the required data actually resides.

3 Platform at a Glance

SLM 4.0 is one control plane organized as seven operating layers, exposing a full arsenal of capabilities over a single managed store set. This section gives one honest paragraph per capability and the cross-system capability coverage map.

3.1 Seven Operating Layers

- L1. Admission.** Identity resolution, scope enforcement, idempotency check, and raw-evidence durability. A canonical write is admitted only after the `ActorContext` is derived from server-side state and the generation fence confirms the idempotency key is not replayed from a superseded profile-binding epoch.

- L2. Queryable Core.** An atomic SQLite transaction writes the canonical fact row, the operation receipt, and the transactional obligation ledger entries for every required projection owner—all or none.
- L3. Enrichment.** Fact extraction, entity resolution, scene construction, temporal provenance derivation, graph edge writing, and optional embedding (mode-dependent).
- L4. Memory Brain.** Behavioral pattern accumulation, feedback and reward recording, consolidation, soft-prompt generation, and guarded skill-evolution workflows.
- L5. Multi-channel Retrieval.** Five candidate producers followed by RRF fusion, optional reranking, and entity-graph score enhancement.
- L6. Context Safety.** Policy evaluation, provenance attachment, redaction of recognized secrets, boundary-marker neutralization, and context budget enforcement.
- L7. Operations.** Lifecycle management, hash-chained audit, exact-match cache and reversible compression, Mesh peer-coordination, backup/restore, and daemon health surfaces.

3.2 Capability Inventory

Multi-channel retrieval. Five parallel candidate producers—dense semantic (with variance-weighted re-scoring), BM25 lexical, temporal (bi-temporal validity filter), Hopfield-associative, and spreading-activation—feed a single-pass Weighted RRF step ($k = 15$) with strategy-adaptive channel weights. An optional cross-encoder reranker runs post-fusion; the entity-graph channel scores the fused candidate set but does not generate independent candidates. Healthy channels participate; degraded channels are excluded with provenance recorded in the response. **What ships is the simplified mode, not the geodesic.** The configuration default selects a variance-weighted Mahalanobis-style re-scoring; the closed-form geodesic on the Gaussian manifold from our prior work is implemented but is not the default path, so a reader inspecting the repository will find the simplified form in production. The contribution that survives is the one that is load-bearing: facts whose observed embeddings have narrower variance—better-confirmed facts—earn a scoring advantage proportional to inverse variance. Calling that a geodesic overstates what executes, and we do not.

Variance-weighted scoring is wired (`retrieval/semantic_channel.py`, `core/engine_wiring.py`).

Information-geometric substrate. The semantic channel applies variance-weighted re-scoring when per-memory variance data is available, falling back to cosine similarity otherwise. Sheaf-cohomology consistency checks and the Riemannian–Langevin lifecycle model were introduced in prior work [20, 21]; the `EbbinghausLangevinCoupling` class is experimental and has no production caller in V4.

Entity knowledge graph. An AST-based code knowledge graph (blast-radius analysis, community detection, execution-flow tracing, incremental indexing with git-hook and file-watcher updates) and an entity store with typed edges and traversal API run inside the managed store. The spreading-activation channel uses this graph to surface multi-hop associations.

Bi-temporal time-awareness. Each fact carries a three-date bi-temporal model (referenced / observation / interval). The `as_of` parameter is threaded across all primary recall surfaces—Python recall engine, HTTP `/recall`, MCP tool, CLI `-as-of`—and drives a temporal validity filter: facts are demoted in ranking under half-open event-validity semantics and transaction-time supersession (*point-in-time demotion*, not snapshot deletion). References are UTC-normalized.

Learning and behavior brain. A zero-LLM behavioral pattern miner captures tool-sequence patterns, correction events, temporal clusters, and cross-project assertions, which a soft-prompt generator distills into a compact context prefix prepended during session-context and auto-recall assembly (ordinary `recall()` ranking is unchanged; initialization failure disables injection fail-soft). Feedback loops (thumbs-up/down, explicit reward signals), outcome recording, a LightGBM ensemble reranker, and a cognitive consolidation queue are all wired and active by default.

Governed skill evolution. SLM 4.0 governs mutation of live agent skill configurations under a `screen` $\rightarrow$ `confirm` $\rightarrow$ `mutate` $\rightarrow$ `blind-verify` $\rightarrow$ `persist` pipeline with candidate provenance, per-cycle budget controls (30-min wall / 10 LLM calls / 3 cycles-per-day, fail-closed on exhaustion), and DB-enforced hash-linked status transitions. *Scope*: the default configuration stops at verified quarantine; an explicit `auto_approve` flag—a configuration gate, not human or RBAC approval—can atomically promote a verified skill into the live loader with rollback on failure. Faithful parent-version lineage and generation tracking are not yet reported.

SLM-Mesh coordination. SLM-Mesh provides serverless, per-tenant-isolated coordination through a local SQLite broker: authenticated peer messages, shared key/value state, and advisory locks with TTL-based expiry and fencing tokens—available in all three operating modes (zero-cloud Mode A through cloud Mode C). Cross-machine state/lock synchronization (opt-in) extends this to LAN peers: leaderless, pull-based last-writer-wins state convergence under (*revision*, *node_id*) and advisory-lock view convergence under fencing-token order, with a durable retrying outbox, optional mDNS discovery, and TLS/pinning/HMAC-signed transport. *Scope*: not linearizable consensus, not quorum replication, not a CRDT database; ordinary storage writes do not invoke the fence primitive; cross-machine coverage is currently the default profile, validated on a two-node loopback.

Unified context optimization. An exact-match cache with tagged invalidation and a reversible compression stage (byte-exact recovery of originals) are co-located in the memory control plane and exposed on three surfaces (proxy, MCP, and skill), with savings telemetry.

Governance-native controls. A governance control plane runs locally and in-process (no network call): a server-derived immutable `ActorContext`; an 18-kind declarative operation policy registry; role-based access (OWNER / ADMIN / MEMBER / VIEWER lattice); multi-scope tenant isolation (personal / shared / global) enforced at write time; GDPR export/erasure with tamper-evident receipts; a hash-chained audit trail; and a deployment-context EU AI Act checklist that abstains from legal classification without intended-use evidence. Details appear in [Section 5](#).

Operational recovery and remediation. Auto-reconciliation redrives failed projection obligations and dead-letters exhausted ingestions; a write-path stall watchdog fast-fails new writers (rather than freezing all tenants) when the single writer stalls; and an OWNER/ADMIN remediation surface—one-click on the dashboard, plus CLI (`slm ops`) and MCP—lists and resolves (retry / force-reconcile / cancel) stuck, dead-lettered, or DEGRADED operations, with failure counts surfaced on `/health`. Detailed in [Section 6.5](#).

3.3 Capability Coverage Matrix

[Table 1](#) maps fifteen capability axes across SLM 4.0 and six representative systems, showing where coverage concentrates and where SLM 4.0’s integration adds axes not individually claimed as novel. Cells are scored from each system’s *public documentation* as of 2026-08-03, not from

private testing: a dash (—) means a capability was *not found in public sources*, which is *not* proof of absence—a system may support more than its docs describe. SLM 4.0’s own ✓ cells are the ones backed by the code and tests in this paper.

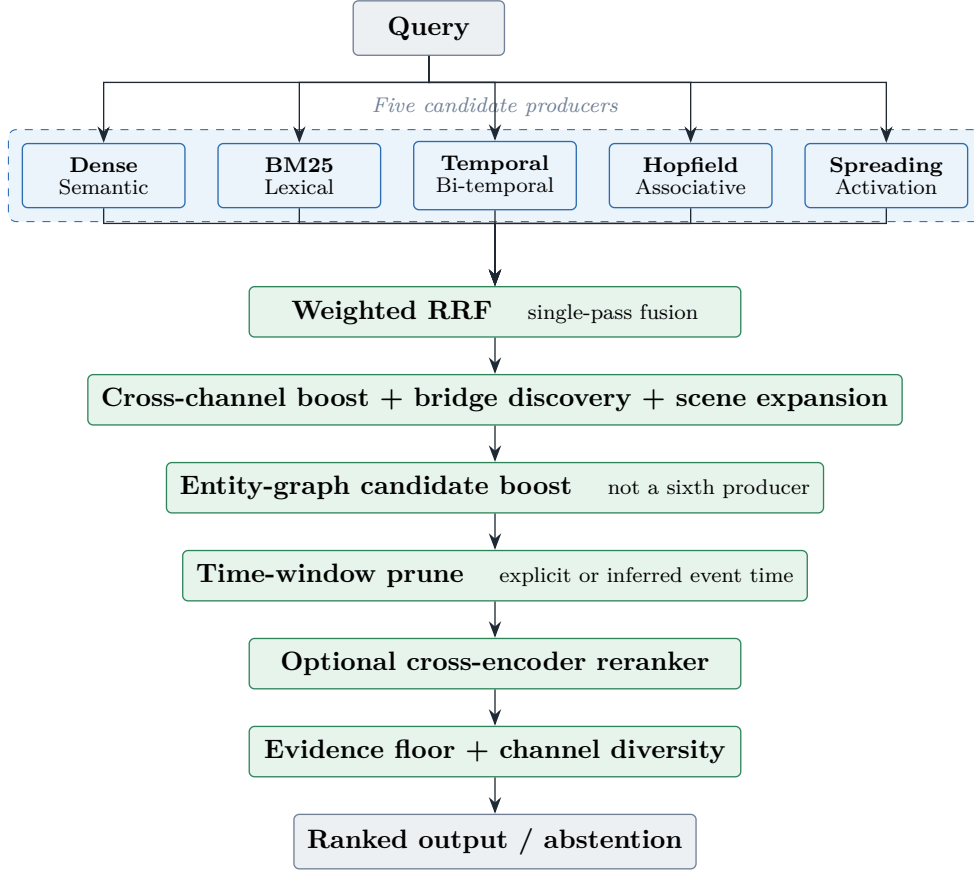


Figure 1: Implemented retrieval order. Five parallel producers feed weighted reciprocal-rank fusion. Cross-channel intersection boosting, code-memory bridge discovery, and scene expansion enlarge or adjust the fused set before entity-graph scoring boosts existing candidates. Event-time pruning precedes the optional cross-encoder. Finally, the evidence floor removes candidates without qualifying primary evidence and channel diversity is enforced before returning ranked results (or an abstention).

4 Platform Architecture

SLM 4.0 is one control plane over many surfaces and stores, governed by the admission invariant stated in [Section 1](#). It is realised through an immutable server-derived `ActorContext`, a declarative `OperationPolicyRegistry`, and a transaction spine whose projection owners each prove their own completion.

System context and boundaries. External actors—human operators via CLI, HTTP, or Dashboard and AI agents via MCP, hooks, or adapters—reach the engine through the daemon bound on port 8765 (`server/unified_daemon.py`). The Admission Gateway derives an immutable `ActorContext` from the authenticated session and evaluates the operation policy registry. In V4 this *full* gateway envelope is wired on the HTTP `/remember` write path and at the internal ingestion admission slot; the other transport surfaces adopt it incrementally—declared MCP tools and selected CLI mutations obtain a shared policy *decision* but not yet the full envelope, and direct legacy, Hook, and Adapter paths remain incremental. [Figure 3](#) is drawn

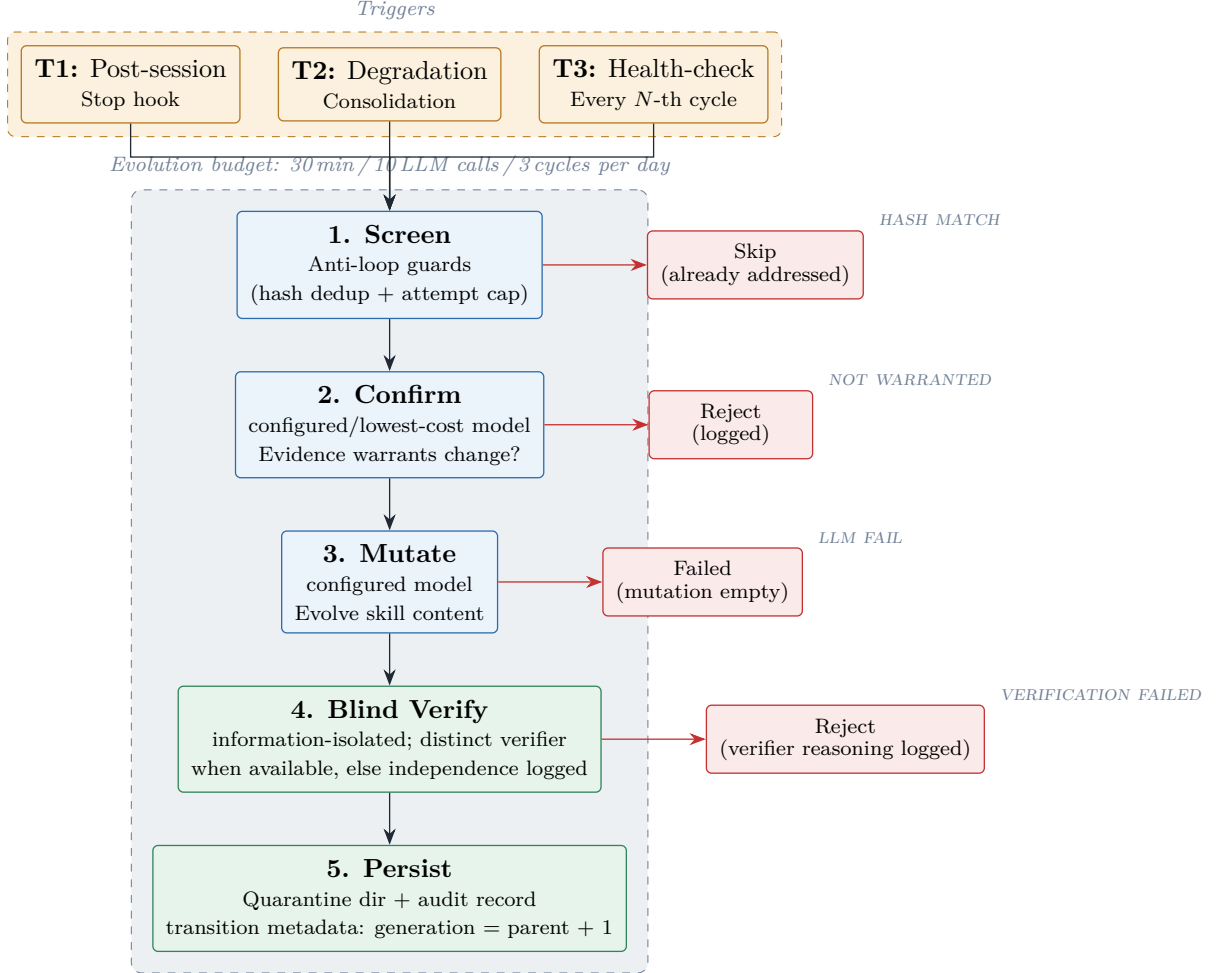


Figure 2: Governed skill evolution pipeline in SLM 4.0 4.0. Three trigger conditions (post-session Stop hook; degradation scan; health-check scan) initiate evaluation within a hard budget envelope. Five guarded stages run sequentially: anti-loop screening, confirmation by the configured/lowest-cost model, mutation by the configured model, blind verification (information-isolated; the verifier cannot see the generator’s rationale or the original skill; a distinct verifier model is used when available, otherwise reduced independence is logged), and persistence to a quarantine directory with transition metadata setting the generation to the latest promoted parent generation plus one. Red exits persist their state-machine transition; promotion audit logging occurs only after blind verification succeeds. Budget exhaustion aborts the cycle before any LLM call is charged beyond the cap.

Table 1: Capability coverage across representative agent-memory systems, scored from *public documentation* as of 2026-08-03. ✓ = documented/supported (for SLM 4.0, code-verified in this paper); ◦ = partial or scoped in public docs; — = not found in public sources (*not* proof of absence). Each system excels on its target axis; this reflects public-documentation focus, not deficiency.

Capability axis	SLM V4	Mem0	Zep/ Graphiti	Letta	Lang- Mem	Super- memory	Memo- base
Local-first, no cloud in write path (Modes A/B)	✓*	◦	◦	◦	◦	◦	◦
Multi-scope isolation + RBAC roles	◦	◦	◦	◦	—	◦	◦
GDPR erasure + hash audit + EU AI Act	◦	◦	◦	◦	—	◦	—
Bi-temporal / time-aware storage	✓	◦	✓	—	—	—	◦
Multi-channel retrieval + RRF fusion	✓	◦	◦	◦	—	◦	—
Knowledge graph + entities	✓	◦	✓	◦	—	—	—
Learning + behavioral brain (soft prompts)	∅ ^w	—	—	—	—	—	—
Governed skill evolution (guarded pipeline)	∅ ^y	—	—	—	—	—	—
Verifiable transactions + manifest	✓	—	—	—	—	—	—
Verified cross-store erasure receipts	◦ ^x	—	—	—	—	—	—
Exact cache + reversible compression	✓	—	—	—	—	—	—
Trusted-peer mesh (serverless, isolated)	◦ ^z	—	—	—	—	—	—
Bounded agentic loops (independent gate)	✓ [†]	—	—	—	—	—	—
Operational recovery + admin remediation (dashboard/CLI)	✓ [†]	—	—	—	—	—	—
Framework adapters + multi-surface API	✓	✓	✓	✓	✓	✓	✓

Notes. Per-cell scoring rationale and primary-source citations are maintained in the full report’s capability ledger; compound axes (e.g. GDPR erasure + hash audit + EU AI Act) are scored on their weakest conjunct. **SLM V4 local-first (✓*)**: local-first by default—full in Modes A/B; Mode C is opt-in cloud. **Evidence tier.** An unmarked ✓ is verified by an experiment in Table 5. A ✓[†] is verified by code inspection and unit tests only—present and reachable, with no fault-injection scenario behind it. A ∅ means implemented and *measured ineffective*: the mechanism exists, is reached, and has not changed an outcome (Section 7). Distinguishing these three is the point of the paper, so the matrix should not flatten them. (^w) **Learning brain (∅)**: instrumented; posteriors at their priors across three stores, so no behaviour has been altered. (^y) **Skill evolution (∅)**: pipeline implemented and gated; zero cycles have completed, so there is nothing to score. Live-loader activation would be configuration-gated, not human- or RBAC-approved. (^x) **Erasure (◦)**: verified over the canonical store and the registered projection owners. Backup artifacts and any remote peer holding state are outside the completeness gate, and the audit chain is same-host tamper evidence rather than a remote attestation. (^z) **Mesh (◦)**: opt-in, default-profile leaderless LWW convergence with fencing-token advisory locks and TLS/pinned transport, two-node-loopback validated; not consensus/quorum/CRDT. All entries scored from public documentation reviewed through 2026-08-03; a — for a competitor reflects its public-doc scope, not a deficiency, and competitors may support more than their docs describe.

from this canonical-path perspective.

Figure 3 shows the overall entry path. Figure 4 shows the transaction spine: the transactional obligation ledger, projection owners, and completion manifest that together enforce projection consistency.

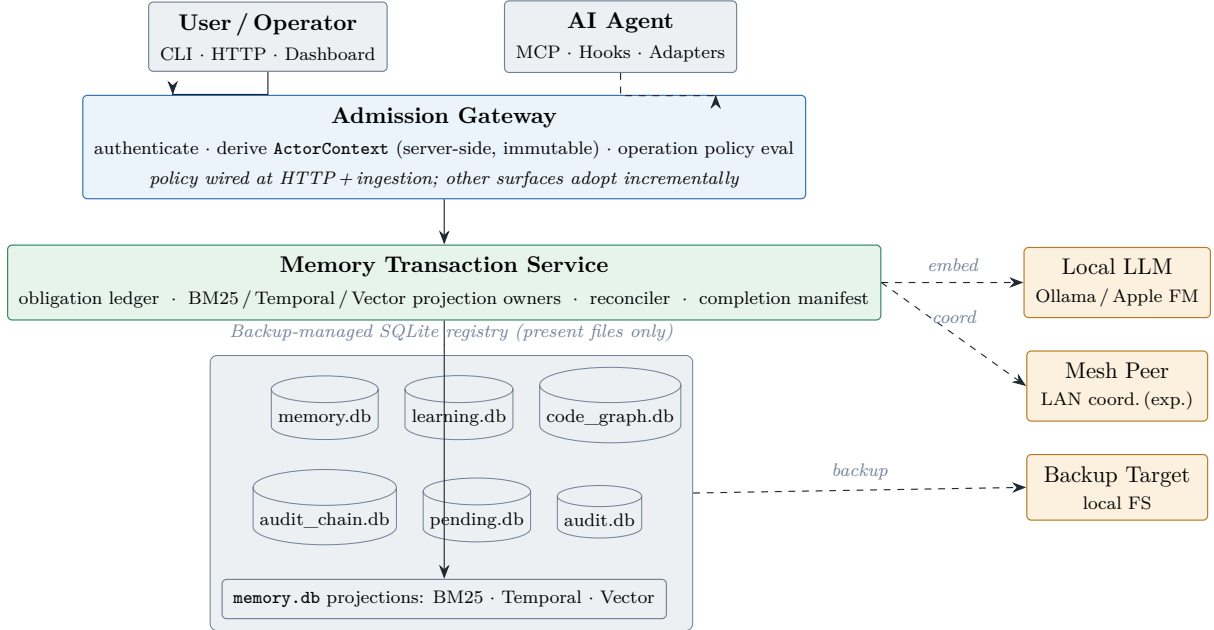


Figure 3: SLM 4.0 system context. The HTTP / internal-ingestion write path reaches the engine through the Admission Gateway (solid arrow from User / Operator); AI agents via MCP / Hooks / Adapters adopt the gateway policy incrementally (dashed arrow); CLI is likewise incremental. The gateway derives an immutable **ActorContext** from the authenticated session and evaluates the operation policy registry. The Memory Transaction Service owns the obligation/reconciliation protocol, not the six database files. The backup registry covers the present subset of **memory.db**, **learning.db**, **audit_chain.db**, **code_graph.db**, **pending.db**, and **audit.db**. Its three registered owners (BM25, Temporal, Vector) maintain projections rooted in the canonical **memory.db**. Dashed arrows indicate optional runtime connections (external systems) or incremental gateway adoption (AI agents).

Store catalog. SLM 4.0 manages six primary SQLite files at a configured data root: **memory.db** (canonical facts, FTS5 indexes, entity/graph tables, vec0 virtual table, mesh coordination tables, RBAC tables); **learning.db** (behavioral signals, reward and outcome tables, LightGBM ranker state, soft prompts, skill-evolution lineage); **audit_chain.db** (append-only hash-chained audit trail); **pending.db** (pending operations queue); **llmcache.db** (exact-match LLM cache); and **admission_journal.db** (encrypted admission journal). The Scale Engine manages two optional derived projections—a CozoDB graph store and a LanceDB vector store—through a **prepare** → **verify** → **promote** → **rollback** lifecycle, with SQLite + sqlite-vec remaining the canonical source of truth and active retrieval backend at all times.

Admission gateway. On the canonical HTTP / **remember** and internal-ingestion paths, the gateway derives an immutable **ActorContext** (principal_id, roles, allowed profiles, active profile_id, transport, and a session-token hash—from the authenticated session, never from the request body) and assembles the operation’s admission inputs (a stable idempotency key, operation kind, resource IDs, scope, deadline, required schema capability, and a redacted payload hash) on the path’s request record—**RememberRequest** for HTTP, **IngestionRequest** for internal ingestion—together with a durable receipt. (An **OperationRequest** dataclass in `core/operation_request.py` defines this envelope as the *target* unified shape; the ship-

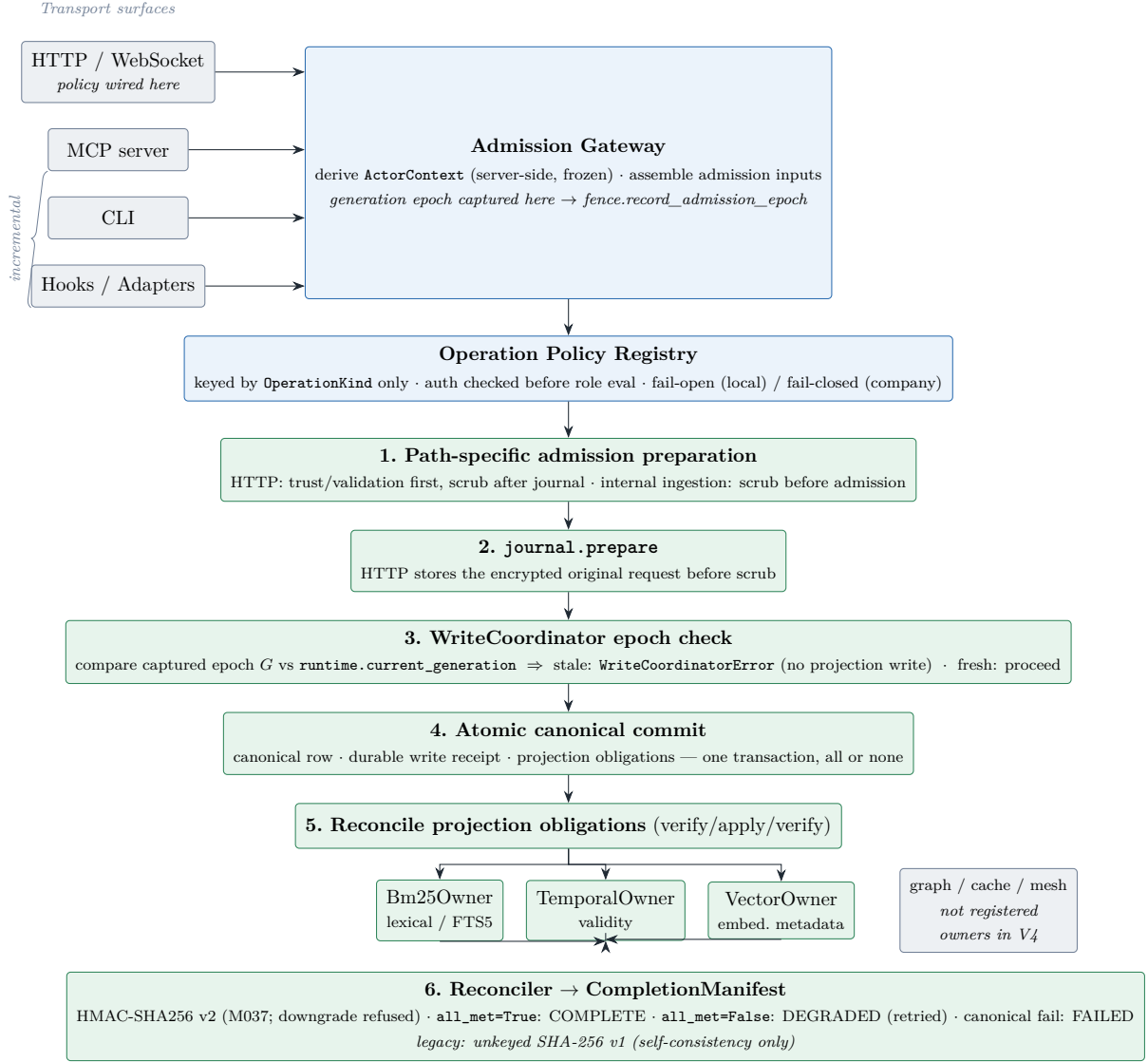


Figure 4: V_4 transaction and policy spine. Transport surfaces reach the Admission Gateway; in V_4 the operation policy is evaluated on the HTTP write path and the internal ingestion admission slot — other surfaces adopt incrementally. The gateway derives an immutable `ActorContext` (server-side; caller-supplied fields are data, not authority) and captures the profile generation epoch in the process-local generation fence. HTTP admission journals the encrypted original before the ingestion writer scrubs it; internal ingestion scrubs before admission. The WriteCoordinator checks the captured epoch before the atomic canonical-row, durable-receipt, and obligation-ledger commit. The Operation Policy Registry is keyed by `OperationKind` only; authentication is checked before role evaluation. Three projection owners are registered in V_4 (`Bm25Owner`, `TemporalOwner`, `VectorOwner`); graph, cache, and mesh stores exist but are *not* transaction owners. The Reconciler emits a hash-checkable `CompletionManifest` (COMPLETE / DEGRADED / FAILED).

ping canonical paths use the split request records above rather than constructing it.) The `ActorContext` is a frozen dataclass (`@dataclass(frozen=True, slots=True)`); no downstream path can mutate it.

Figure 5 shows the admission and fence protocol for a canonical `/remember` call.

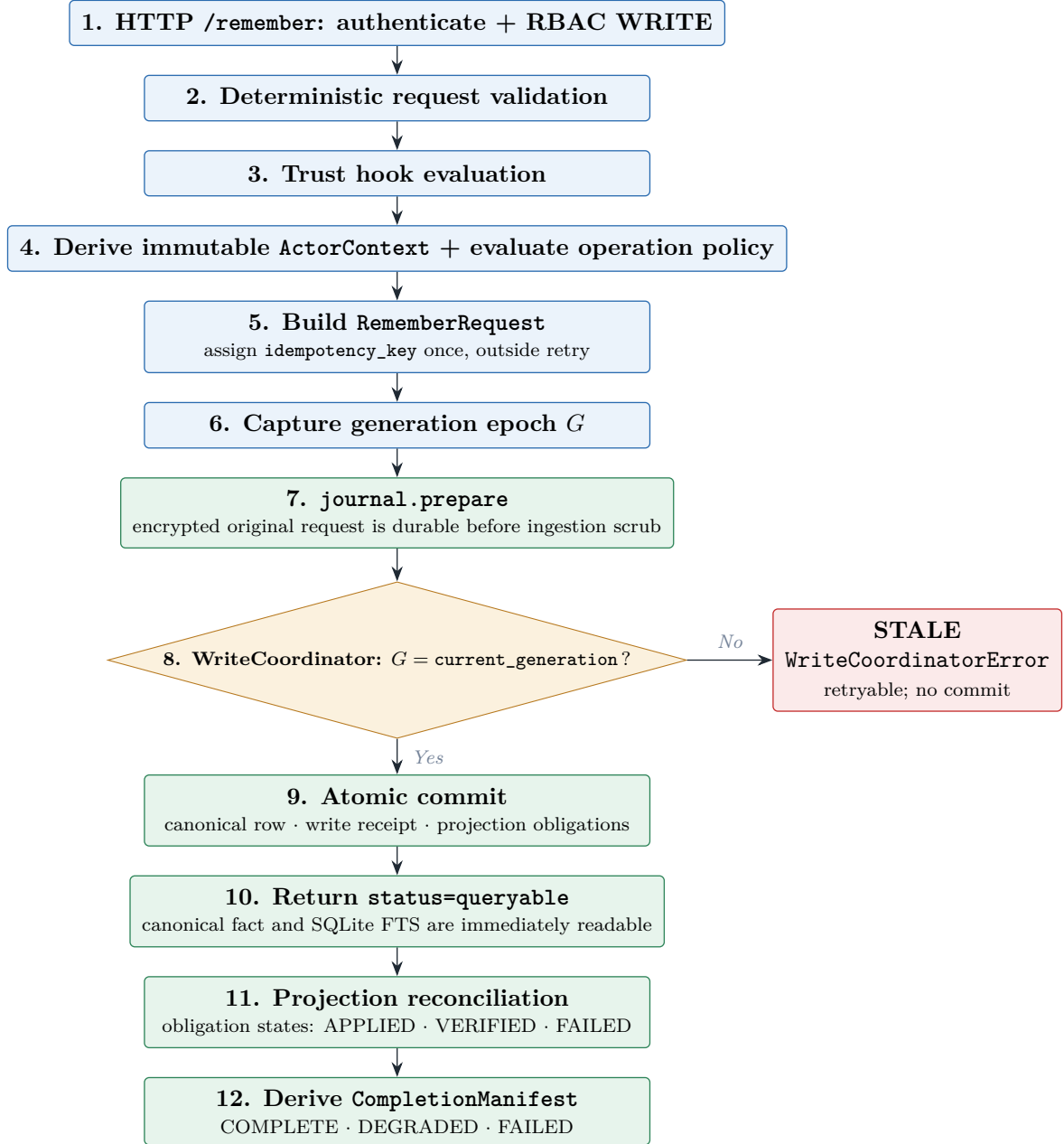


Figure 5: HTTP `/remember` admission sequence. Authentication and WRITE authorization precede deterministic validation and the trust hook. The server then derives authority in an immutable `ActorContext`, evaluates the operation policy, builds one idempotent `RememberRequest`, and captures generation `G`. The encrypted original request is journaled before the ingestion writer scrubs it. The writer checks `G` before one transaction commits the canonical row, durable receipt, and projection obligations. The caller receives `status=queryable`; projection reconciliation subsequently records APPLIED, VERIFIED, or FAILED evidence.

Delivery surfaces. The daemon hosts the HTTP REST API, dashboard, and HTTP MCP endpoint in a single FastAPI application. The MCP server serves both HTTP and stdio transports; the CLI runs as a separate process with canonical `remember` proxying to

the daemon. Nine separately packaged framework adapters implement each framework’s native memory interface backed by the local SLM engine: LangGraph (`BaseStore`), Semantic Kernel (`VectorStore`), Microsoft Agent Framework (`ContextProvider`), LangChain (`BaseChatMessageHistory`), LlamaIndex (`BaseChatStore`), CrewAI (`StorageBackend`), AutoGen (`Memory`), Google ADK (`BaseMemoryService`), and OpenAI Agents (`SessionABC`).

Three operating modes. Mode A (fully local, zero cloud: nomic-embed-text-v1.5 768-d, deterministic rule extraction, local PyTorch cross-encoder reranker). Mode B (local Ollama LLM: Phi-3 / Llama 3.2, local embeddings, local PyTorch reranker). Mode C (cloud text-embedding-3-large 3072-d, configured cloud LLM, Cohere reranker; content is sent to the configured provider). Every mode reports `eu_ai_act_compliant=None`; intended use and deployment context are required for legal classification. Mode A is the default at installation; “no cloud in the memory path” applies to Modes A and B, not universally.

5 Governance Control Plane

The strategic premise of SLM 4.0 V4 is *rent the LLM, own the memory*. Governance is not retrofitted—it is the substrate on which canonical operations run. The controls in this section execute locally and in-process, without a network call. Memory *processing* is fully local in Modes A and B; the opt-in Mode C may send configured embedding, reranking, extraction, or answer payloads to a provider, while governance state and decisions remain local.

Governing invariant (scope). V4 realises the full envelope (one authenticated actor, one profile generation, one policy decision, one durable receipt, one verifiable completion state) on the canonical HTTP `/remember` path and internal ingestion, over the three registered projection owners; declared MCP tools and selected CLI mutations additionally obtain a shared policy *decision* but do not yet share that operation envelope or receipt; direct legacy mutations and HOOK/ADAPTER surfaces remain incremental. We present it as the enforced design of the canonical write path and the target for the platform, not as a property already enforced on every surface.

5.1 Server-Derived Actor Identity

`ActorContext` (`core/actor_context.py`) is a frozen dataclass that carries the complete, server-resolved identity for a single operation. Its fields are populated exclusively from server-authenticated state—session tokens, daemon descriptors, the profile runtime, and RBAC results—**never from the request body**. Key fields: `principal_id` (stable user or operator identifier, never a raw session token); `roles` (a `FrozenSet` drawn from the lattice `OWNER ≥ ADMIN ≥ MEMBER ≥ VIEWER`, plus `SYSTEM` and `ANONYMOUS`); `session_token_hash` (first 16 hex chars of the SHA-256 of the raw token, for audit attribution only; no raw session material is retained on any path); `transport` (HTTP / MCP / CLI / MESH / HOOK / INTERNAL / DASHBOARD / ADAPTER).

5.2 Operation Policy Registry

`OperationPolicyRegistry` (`core/operation_policy_registry.py`) is the declarative admission layer. `evaluate()` is *pure and CPU-only*—no file, no network, no database access—completing in microseconds. The internal policy table is a `types.MappingProxyType`, preventing external mutation after construction. One registry makes “the same evaluator and policy table on each participating surface” a checkable property—any caller that invokes it obtains identical evaluation semantics—rather than universal surface coverage, which the registry alone does not establish.

The `evaluate(kind, actor, mode)` chain runs an ordered 8-step decision sequence: resolve operation kind; check the explicit deny list; look up the registered `OperationPolicy` (fail-open

in local modes, fail-closed in company/remote modes for unknown kinds); empty-roles check (deny all); **authentication before authorisation**; role disjoint check; transport check; payload size annotation (never a rejection). The registry contains 20 operation kinds; Table 2 shows a representative subset.

Figure 6 illustrates the evaluation chain.

Operation	Required roles	Allowed transports	Audit
REMEMBER	OWNER, ADMIN, MEMBER	all	standard
RECALL	all (incl. VIEWER)	all	none
FORGET / CORRECT	OWNER, ADMIN	all	full
ERASE	OWNER only	all	full*
BACKUP	OWNER only	all	standard
RESTORE_BACKUP	OWNER only	all	full
MODE_CHANGE	OWNER only	admin transports	full
SCHEMA_MIGRATE	OWNER only	CLI / INTERNAL only	full
MESH_SEND	OWNER, ADMIN, MEMBER	mesh transports	standard
EVOLVE_SKILL	OWNER, ADMIN	all	full

Table 2: Representative policy table from `OperationPolicyRegistry.default()`. The full table contains 20 operation kinds. *ERASE additionally sets `resource_ownership_check=True` and `redaction_policy="full"`.

5.3 Multi-Scope Isolation and Profile Generation Fence

SLM 4.0 separates two orthogonal isolation axes: the *profile* (individual / team / company namespace) and the *scope* (**personal**, **shared**, or **global**) within and across profiles. Every tenant-owned memory table carries a `profile_id` column as a partition key; cross-scope access is *default-deny*. A fact in **personal** scope is not visible to a request scoped to **shared** unless the writer granted it explicitly—enforced at write time (scope label committed with the fact) via profile-scoped predicates across all five retrieval channels.

The generation fence rejects a stale, superseded-epoch admitted *write* on the canonical remember path; it is not the retrieval-channel isolation mechanism. Scope of isolation claim: authorization-layer control, not cryptographic multi-tenancy; a single OS user with direct disk access is outside the threat model.

5.4 GDPR Export and Erasure

GDPRCompliance (`compliance/gdpr.py`) implements three GDPR data rights locally—no request leaves the device. The Right to Erasure (Art. 17) `forget_profile` proceeds in three ordered phases:

1. **Audit before deletion (Art. 5(2) accountability).** The runtime writes an erasure-request record to the hash-chained `audit_chain.db` *before* any row is deleted. This is a *hard precondition*: if the audit write fails, the erasure fails *closed*—it aborts and deletes nothing, so no erasure ever occurs without an accountability record.
2. **Cache and vector purge.** The brain cache DB is purged before main-DB deletions; the vector store’s `delete()` is called per fact ID.
3. **Schema-driven wipe.** Every discovered profile-scoped table is deleted in a single pass. After all deletions, the runtime re-counts residual rows and distinguishes *deletion* from *provable deletion*, and carries the two as separate flags. Most systems report the first. A deletion that succeeded but cannot be demonstrated afterwards is a different object, for a regulator, from one that can, and conflating them is what makes an erasure claim

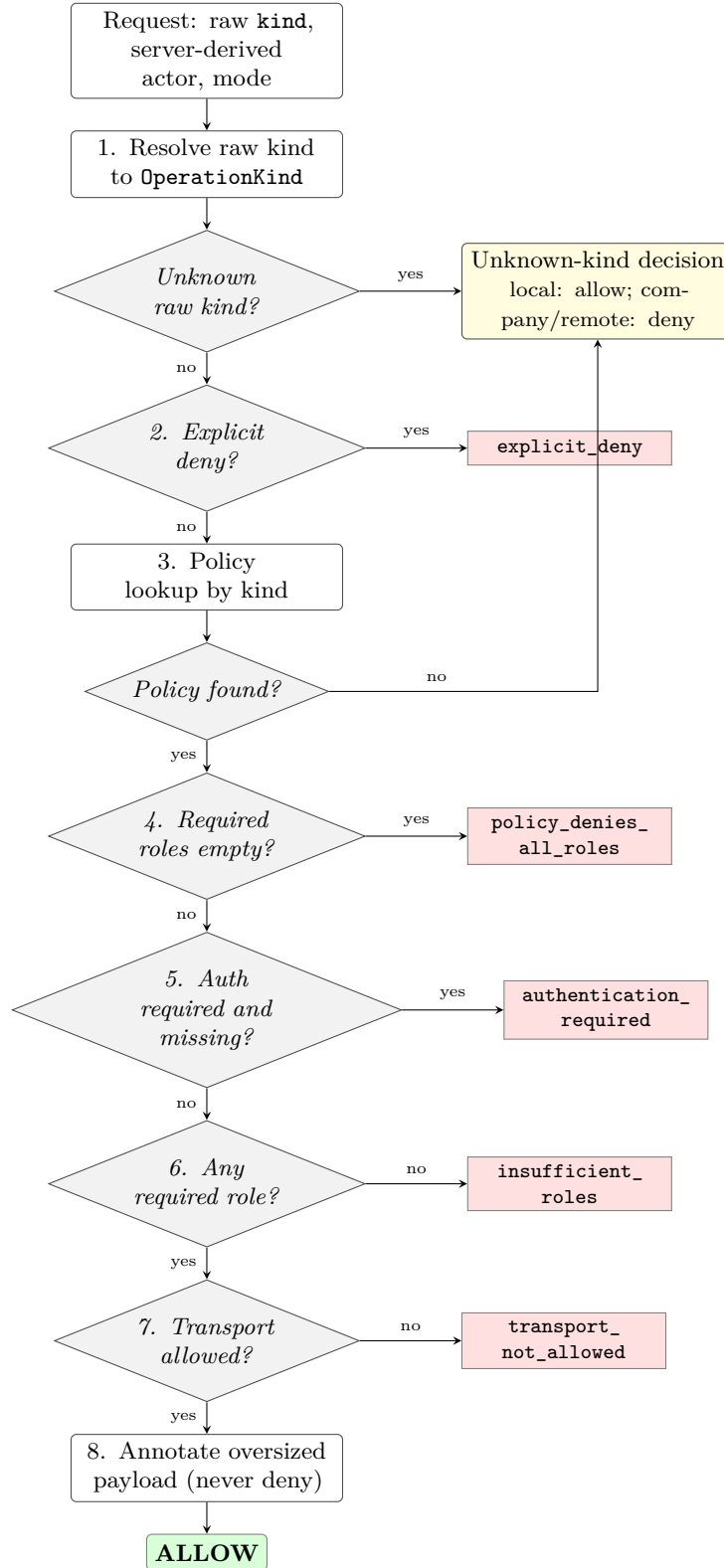


Figure 6: **RBAC decision flow** in `OperationPolicyRegistry.evaluate()`. Kind resolution and the mode-dependent unknown-kind decision precede explicit deny and policy lookup. A registered policy with no required roles denies all actors. Authentication is tested before role authorization, followed by transport. Payload size is advisory: oversized input annotates an otherwise allowed decision.

unfalsifiable. The operation is also fail-closed on its own audit precondition: if the tamper-evident chain cannot be written, the erasure does not proceed, so there is no state in which data is destroyed and the record of destroying it is missing—which is the state Article 5(2) accountability cannot survive. The wipe is driven from the schema rather than an enumerated table list, so a table added later is covered without anyone remembering to add it; compliance drift by omission is designed out. It returns an explicit `erasure_complete` flag, set only when no residue remains and no per-table delete, residue re-count, context-cache purge, learning-DB reset, vector-purge, or owner-proof failure occurred—any such error, including a top-level purge exception or an uncountable table, forces `erasure_complete=0` (fail-closed reporting). Backup artifacts and remote peers are outside the erasure scope (future work).

5.5 Hash-Chained Audit Trail and EU AI Act Deployment Checklist

`AuditChain` (`compliance/audit.py`) is a tamper-evident hash-chain audit log in its own SQLite file (`audit_chain.db`), completely independent of the main store. The chain hash is SHA-256 over the prior hash and each entry’s fields; verification walks every entry in insertion order and returns `False` on the first mismatch. A sibling sidecar (an out-of-table anchor recording chain length and head hash) detects DB-only suffix or whole-chain truncation *when the sidecar was successfully written and remains intact*; it is a same-host integrity aid, not a remote trust anchor, and an anchor-write failure is logged rather than blocking the audited write.

`EUAIActChecker.check_compliance(mode)` produces a frozen assessment record describing observable mode capabilities and missing deployment context. For every mode it returns `compliant=None`, `risk_category="undetermined"`, and unset transparency and human-oversight status until intended use, actor roles, and deployment evidence are supplied. Local versus provider-assisted processing alone cannot determine an EU AI Act risk category. Two dates matter for a reader positioning a deployment against it, and both fell before this paper’s first version: obligations for general-purpose model providers took effect on 2025-08-02, and the Commission’s enforcement powers over those providers—requests for information, model access, and recall—became applicable on 2026-08-02, alongside the Article 50 transparency duties, whose enforcement sits with national market-surveillance authorities. The high-risk obligations originally scheduled for that same date were deferred to 2027-12-02 and did not take effect. The checklist records deployment context and does not track this calendar; a reader must confirm the current position independently. This is an engineering checklist, not a conformity assessment or legal certification.

6 Reliability Spine

The governance layer answers *who may act and on which scope*. This section answers *how we guarantee that the action succeeds consistently*. Four mechanisms form the V4 reliability spine: generation-fenced admission, verifiable memory transactions, cross-store verified erasure, and bi-temporal as-of. SLM-Mesh provides the coordination layer. Together they realise the governing design goal on the write path.

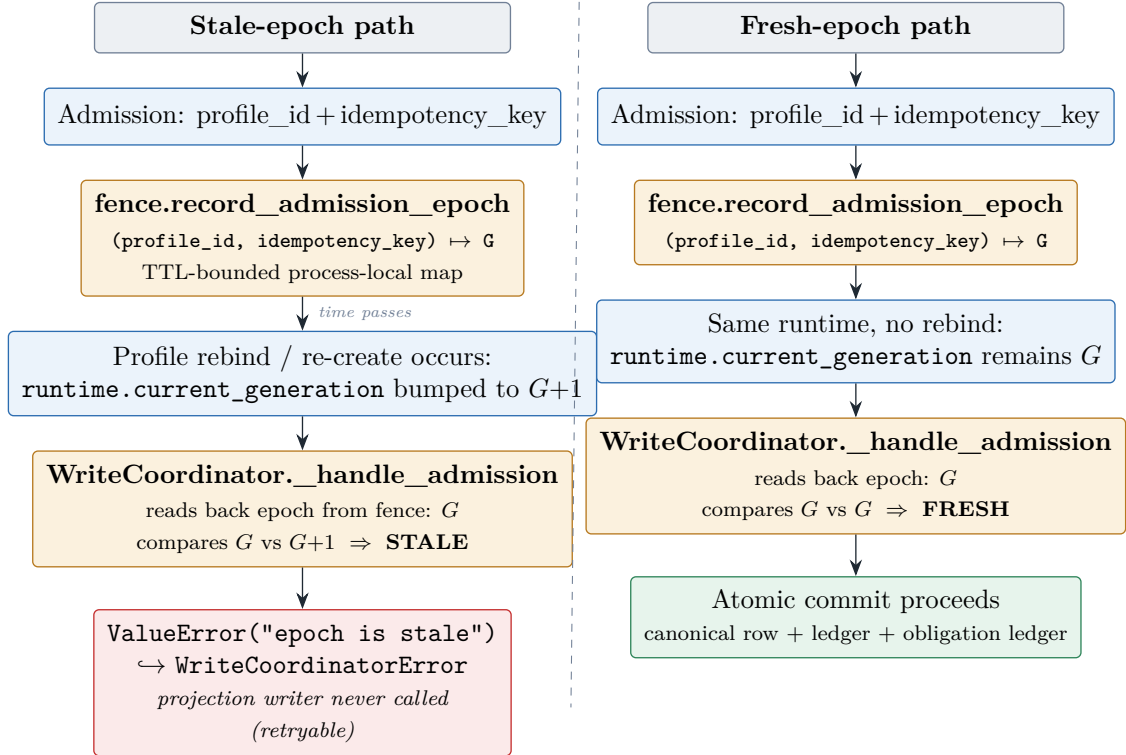
6.1 Generation-Fenced Admission

A profile deleted and recreated with the same ID must not allow a cached context to write into the new profile’s data. The generation fence uses a process-local, `threading.RLock`-protected, TTL-bound map of `(profile_id, idempotency_key) → (epoch, timestamp)` in `storage/generation_fence.py`, with entries expiring after 300 seconds.

At admission, `record_admission_epoch` stores the current profile generation for the idempotency key. If two concurrent admissions record different epochs for

the same key, the fence sets the entry to `_CONFLICT_EPOCH = -1`—a sentinel that can never equal a real generation (≥ 0)—so both requests are subsequently rejected. When `CanonicalRememberRuntime._handle_admission` processes the write, it calls `admitted_epoch()` and compares against the runtime’s current generation; a mismatch raises `ValueError("epoch is stale")`, which the `WriteCoordinator` wraps as `WriteCoordinatorError` *before* any projection writer is called. The generation epoch is server-derived and cannot be supplied by a caller.

Figure 7 shows the fence protocol. *Scope*: in-process, TTL-bounded, not SQLite-backed; not a revision CAS; does not survive process restart. The canonical HTTP `/remember` and internal-ingestion boundaries synthesize a non-empty idempotency key (a UUID) when the caller omits one, so omission at those public boundaries does not bypass the fence.



The generation epoch is server-derived and cannot be supplied by a caller. ActorContext.active_profile_generation defaults to 0 for HTTP actors; the fence is independent of that field and relies solely on the (profile_id, idempotency_key) → epoch map (storage/generation_fence.py).

Figure 7: Profile generation fence (exp7). The fence is a process-local, TTL-bounded map of $(\text{profile_id}, \text{idempotency_key}) \rightarrow \text{epoch}$. At admission, `record_admission_epoch()` stores the current generation G . If the profile is deleted and recreated (or otherwise rebound) before the write coordinator runs, the runtime’s `current_generation` advances to $G+1$. `_handle_admission` reads back G from the fence and compares it to the current generation: a mismatch raises `WriteCoordinatorError` before any projection writer is called (retryable). On a matching generation the atomic commit proceeds normally. The fence does *not* perform per-owner revision CAS; it rejects stale-epoch admissions at the coordinator level.

6.2 Verifiable Memory Transactions

Every canonical remembered-fact write must produce consistent state across three physical representations: BM25 token index, temporal store, and vector projection. V4 routes the

canonical remember path through a transactional obligation ledger with per-projection ownership and a hash-sealed completion manifest; registry-gated MCP/CLI operations and direct legacy mutations do not record these obligations.

Obligation ledger. On every canonical remembered write, `_record_projection_obligations` creates obligation records for the three registered owners ("bm25", "temporal", "vector") in `projection_obligations` with `state=PENDING`. This is the V4 transactional obligation ledger—distinct from the Mesh remote-delivery outbox.

Projection owners. Each of the three owners (`Bm25Owner`, `TemporalOwner`, `VectorOwner` in `core/transactions/concrete_owners.py`) implements the `ProjectionOwner` protocol: `apply` / `verify` / `compensate` / `erase` / `prove_erased` / `health`. Their checksums are SHA-256 over owner name, sorted fact IDs, and per-fact fingerprints.

Completion manifest. After all owners have been applied and verified, `CompletionManifest` assembles an `OwnerEvidence` record per obligation and seals the canonical JSON serialisation: on M037-capable databases¹ with an installation-key-derived HMAC-SHA256 (version 2, refusing a version-1 downgrade at verification); on older schemas with a deterministic unkeyed SHA-256 (accidental-corruption detection only, not tamper-evidence against a malicious DB writer). State derives as: `COMPLETE` (canonical write succeeded, all owners in terminal success); `DEGRADED` (canonical write succeeded, at least one owner failed, compensation ran); `FAILED` (canonical write itself failed). A `DEGRADED` manifest is never silently promoted to `COMPLETE`.

The reconciler maintenance loop redrives pending or missing-manifest obligations (capped at 10 attempts per obligation), bounding eventual drift from transient write failures.

Figure 8 shows the transaction spine. Figure 9 shows the manifest state machine.

6.3 Cross-Store Verified Erasure

`ErasureService` (`core/transactions/erasure.py`) coordinates verified deletion across all registered projection owners:

1. **Obligation recording.** One `ERASE` obligation per owner enters `projection_obligations` in `PENDING` state.
2. **Tombstones.** Before per-owner deletion, a tombstone row is inserted per fact ID. A provenance conflict (two erasure operations recording different `memory_id` values for the same fact) fails the operation *closed*: `remove()` aborts before any owner deletion and the receipt is marked `FAILED`.
3. **Per-owner erase.** Each owner's `erase(context)` removes its physical entries and returns an `OwnerErasureProof` with an erasure checksum.
4. **Proof verification.** `ErasureService.finalize()` calls each owner's `prove_erased(context)`—a live physical re-query of the underlying tables, not a re-hash of the envelope—to re-confirm absence, then seals the full receipt with an installation-key-derived HMAC-SHA256 on M037-capable databases (version 2, refusing a version-1 downgrade) or unkeyed SHA-256 on older schemas.

The receipt carries `all_erased=True` only when every owner's proof reports `erased=True`; partial residue sets `all_erased=False`. The three registered owners in V4 are `Bm25Owner` (BM25 tokens), `TemporalOwner` (temporal validity rows), and `VectorOwner` (embedding metadata). Backup artifacts and remote peers are not erased and are outside the current scope.

¹M037 denotes schema migration 037 (schema version 37), which added the per-installation key column from which the manifest seal's HMAC-SHA256 key is derived.

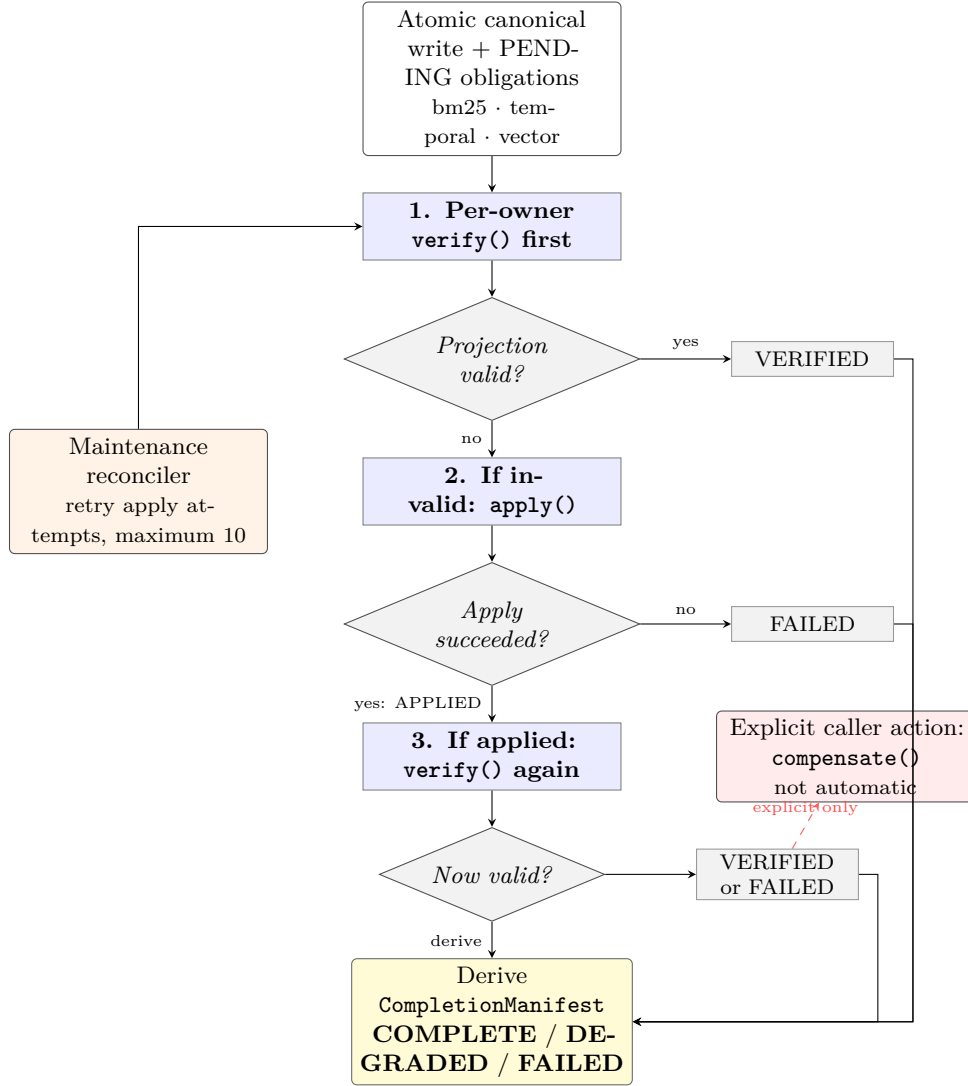


Figure 8: **Verifiable memory transaction spine.** Each reconciliation attempt verifies the existing projection first. Only an invalid projection is applied, and a successful apply is verified again before the obligation is marked VERIFIED; failed apply or verification records FAILED. Maintenance retries are idempotent and apply attempts are bounded at ten. Compensation is a separate explicit service call, not an automatic consequence of failure. The manifest is derived from canonical presence and current obligation evidence.

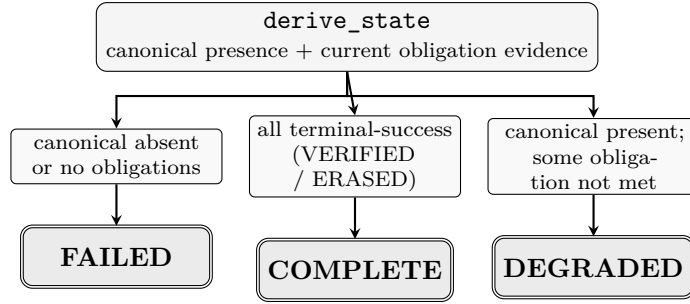


Figure 9: **Completion-manifest state derivation.** A reconcile cycle recomputes exactly one output from canonical presence and the current obligation set: **FAILED** when the canonical commit is absent or the set is empty, **COMPLETE** when every obligation is terminal-success (VERIFIED or ERASED), and **DEGRADED** otherwise. The fan-out depicts mutually exclusive derivation conditions; there are no state-to-state transitions.

6.4 SLM-Mesh: Coordination Across All Three Modes

SLM-Mesh provides serverless, per-tenant-isolated coordination through a local SQLite broker (in `memory.db` via `mesh_*`-prefixed tables): authenticated peer messages, shared key/value state, and advisory locks with TTL-based expiry and fencing tokens. Eight mesh operations are exposed as MCP tools. The broker is available in all three operating modes, including fully offline Mode A.

Per-tenant isolation. Server-assigned principals are bound to a `profile_id`; `check_cross_profile_sender()` rejects any peer ID that belongs to a different profile. Cross-machine state/lock synchronization currently covers the default profile only.

Authentication and fencing. When the broker binds to a non-loopback host it requires `SLM_MESH_SHARED_SECRET`—the broker raises `RuntimeError` at construction time if the host is not localhost and the secret is absent. Fencing-token order resolves advisory lock *views* to a single holder; ordinary storage-mutation paths do not currently call that primitive, so this is advisory coordination, not end-to-end single-writer enforcement over storage writes.

Cross-machine coordination (opt-in). Each node periodically pulls a peer’s deltas and merges them deterministically: shared state converges by last-writer-wins under the total order (*revision, node_id*); advisory-lock views resolve under (*fencing_token, node_id*). A durable remote outbox provides bounded durable retry with duplicate possibility (jittered-backoff retry, 48-hour TTL, per-peer cap, dead-letter path); delivery is *not* guaranteed after cap eviction, enqueue failure, retry exhaustion, or 48-hour expiry. The transport rejects production plaintext and supports verified TLS, optional pinned certificate, bearer authentication, and HMAC-signed messages; opt-in mDNS discovery is off by default.

Figure 10 shows the mesh topology. *Scope*: cross-machine features are opt-in; WAN-scale operation, per-device revocation, and fault-injection at scale are future work. SLM-Mesh provides leaderless LWW convergence with fencing-token advisory locking—**not** linearizable consensus, not quorum replication, and not a general CRDT merge.

6.5 Operational Recovery and Admin Remediation

A shared deployment must ensure that a stuck or failed operation for one tenant neither corrupts nor silently blocks the others, and that every failure is either auto-recovered or resolvable by an operator. V4 layers three mechanisms on the transaction spine.

Auto-recovery. The maintenance reconciler redrives pending and failed projection obligations (capped at ten attempts per obligation); the ingestion path retries a materialization up to ten times and then records the operation in a durable `dead_letter_operations` table

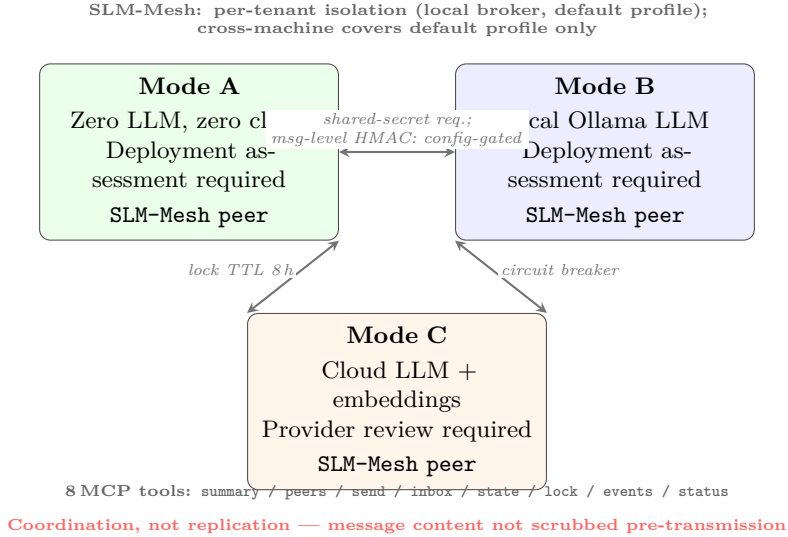


Figure 10: **SLM-Mesh topology across all three operating modes.** A mesh instance runs alongside any SLM daemon regardless of mode: Mode A (zero cloud), Mode B (local LLM), or Mode C (cloud). Non-loopback binding requires a shared secret (`SLM_MESH_SHARED_SECRET`) to start; message-level HMAC signing/verification is optional and configuration-dependent (off by default for compatibility). Distributed locks auto-expire after 8 hours so a crashed peer cannot deadlock a resource. A circuit breaker on `mesh_send` fast-fails after three consecutive daemon-unreachable errors and re-probes after a 60 s cooldown. Per-tenant isolation—server-assigned principals bound to `profile_id`—is inherited by every local-broker mesh primitive; cross-machine state and lock synchronisation currently covers the default profile only. Scope: coordination (signals, shared state, locks), not a conflict-resolving replicated database.

with its content, error, and attempt count. Transient failures heal without human action, and a permanently failed operation is retained rather than lost.

Stall isolation (write-path watchdog). The single-writer `WriteCoordinator` serializes canonical writes for correctness. To keep one pathological operation from freezing every tenant’s writes, each in-flight item is watched: if the worker exceeds a stall threshold (default 30 s, far above the 1 s per-item deadline) on a single item, the coordinator trips a circuit breaker so that *new* submissions fail fast with an explanatory `WriterStalledError` (“write subsystem stalled; admin remediation required”) instead of queuing behind a dead worker; the breaker clears automatically once the item completes. Callers receive an immediate, actionable error rather than a silent hang. *Scope:* the watchdog surfaces and fast-fails a stall but does not forcibly terminate the worker thread—a hung native handler still requires a restart (future work).

Surfacing and one-click remediation. Failures are made visible rather than buried in logs: the daemon `/health` and `get_status` surfaces report `dead_letter_count`, `degraded_operations`, and `writer_stalled`. An OWNER/ADMIN operator can enumerate and resolve them through one remediation surface exposed uniformly on the dashboard, the CLI (`slm ops list / slm ops resolve`), and MCP (`list_failed_operations`, `resolve_operation`), gated by the `OPS_INSPECT/OPS_RESOLVE` policy kinds with a full audit record on resolve. `resolve` supports three actions on a specific operation—*retry* (re-enqueue a dead-lettered ingestion), *force-reconcile* (redrive its projection obligations immediately, bypassing the periodic throttle), and *cancel* (mark it terminally handled). Because a large share of SLM 4.0 operators are non-technical, the dashboard presents this in plain language—a health banner and a “needs attention” table with per-row Retry / Re-sync / Cancel buttons—so recovery requires neither the CLI nor raw SQL. These paths are covered by dedicated tests (stall-watchdog fast-fail and recovery, RBAC-gated listing, and resolve semantics).

7 The Learning Layer: Measuring Whether a Learner Learns

7.1 An asymmetry between the write path and the behaviour path

Section 6 states the invariant the write path enforces: one authenticated actor, one profile generation, one policy decision, one durable receipt, one verifiable completion state. On that path, a fact cannot enter the canonical store without satisfying all five. Other transports adopt the gateway incrementally (Section 4), so this is a property of the primary write path and not yet of every route into the store.

Nothing comparable governed the path that changes the agent’s *behaviour*. A reward could move a channel-selection posterior, a posterior could reweight retrieval, and a soft prompt could enter a session prefix, with no admission decision, no receipt, and no completion state anywhere in that chain. We described the result in the previous version of this paper as a governed learning and behaviour brain. It was neither governed nor learning, and the second half is measurable.

7.2 Three Bayesian learners, none of which has learned anything

Each of the three mechanisms below is implemented, is reached on a live call path, and has never acquired a preference between the options it exists to choose among. The numbers were derived directly from the stores rather than from any monitoring surface.

These figures come from the authors’ own operating stores, which a reader does not have. They are not harness output and are not reproducible from the repository; they are a frozen reading taken at a stated instant (2026-08-23T18:46:39Z) from stores that continue to accumulate, so a later reading of the same stores will differ—the released evidence bundle contains an earlier snapshot taken the same evening at 17:55Z whose counts are correspondingly lower, and the gap between them is ninety minutes of ordinary operation rather than a discrepancy. The re-derivation queries are released with the evidence bundle so the method is checkable even where the data is not. What *is* reproducible by anyone is the mechanism: both invariants of Section 7.4 ship in the package, run against any store, and will report the same verdicts on any deployment in the same condition.

The channel-selection bandit. Thompson sampling over Beta posteriors, one arm per retrieval-weight configuration, updated as $\alpha += r$, $\beta += (1 - r)$. Across three independent stores—a live working store, an archived export taken a month earlier, and an unrelated instrumented store—every arm satisfies

$$\alpha - \alpha_0 = \beta - \beta_0 = n/2 \tag{1}$$

exactly, on **459 of 459 arms** over **5,657** recorded plays.

What Equation (1) establishes needs stating precisely, because it is weaker than it first looks and still sufficient. Since $\alpha - \alpha_0 = \sum_i r_i$ and $\beta - \beta_0 = n - \sum_i r_i$, the identity says only that $\sum_i r_i = n/2$: the rewards *average* exactly one half. It does *not* follow that each individual reward was 0.5—an alternating stream of zeros and ones satisfies it identically, and so does any symmetric mixture. So the identity alone does not prove the reward channel was emitting a constant.

It proves something sufficient anyway, though not what we first said it proved. Every arm’s posterior mean is exactly 0.5, the prior mean, so no arm has acquired a preference over any other. We previously wrote that a Thompson sampler in this state “selects uniformly at random forever”. That is wrong, and we keep the error here because it has the same shape as everything else in this section.

A neutral update does not leave a posterior where it started. Both parameters grow, so Beta(1+n/2, 1+n/2) holds the mean at one half while the variance falls. Arms are played different numbers of times, so their posteriors concentrate by different amounts, and a sampler

drawing one value from each does not choose among them evenly. Drawing 10^6 samples from Beta(1, 1), Beta(10, 10) and Beta(100, 100)—which share a mean of exactly one half—the argmax falls to them in proportion 0.454/0.291/0.255.

So the learner is not standing still. It is converging, with increasing confidence, on the conclusion that every arm is average, and what remains of its exploration is steered by how often an arm happened to be played rather than by anything it observed. We think that is worse than a frozen learner, and we did not see it until someone checked the arithmetic rather than the claim.

And the exactness is itself the evidence. A genuine reward stream would have to land on $\sum_i r_i = n/2$ to the last representable digit on all 459 arms independently, across three stores with different play counts. Where per-observation data exists we can close the gap directly rather than by inference: the source-trust learner’s observation table records **629** rows and its distinct reward set is exactly $\{0.5\}$ —every single observation is the neutral fallback. The bandit has no comparable per-play record, and that absence is itself the second finding below.

The settlement ledger holds 4 rows against those 5,657 arm-plays, and none is settled. The two counters are not inconsistent: a play increments its arm and separately writes a ledger row that a later authenticated outcome resolves. The row-writing was disabled at the only caller on 2026-07-27, together with the per-exposure signal enqueue it shared a flag with—the same date on which the arms’ last-played timestamps stop. Nothing was written for the reward path to settle, so every play applied the neutral prior and none was ever revised.

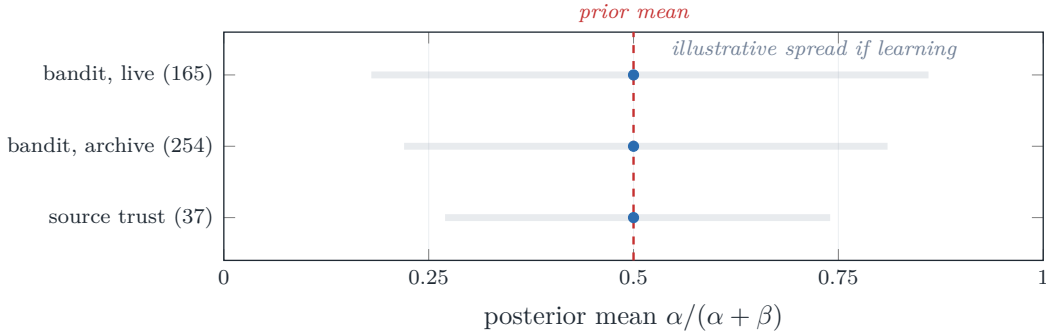


Figure 11: Three Beta populations, and the entire observed range of each. Every unit in all three sits at posterior mean exactly 0.5—the prior mean—so each population’s range is a single point rather than an interval. Live bandit: 165 arms over 1,405 plays. Archived bandit from a store captured a month earlier: 254 arms over 1,394 plays. Source trust: 37 sources over 629 observations whose recorded reward is 0.5 in every row. The shaded bands show, for scale only, where a population receiving differential signal would spread; no measurement is plotted there. Frozen reading, 2026-08-23T18:46:39Z, from the authors’ own operating stores (Section 7.2); these stores continue to accumulate and a later reading will differ.

The source-trust model. A second Beta posterior, one unit per evidence source. Identical signature: 37 of 37 sources at posterior mean 0.5, across **629** observations whose recorded reward is 0.5 in every single row. This learner was still writing observations during the preparation of this section.

Trust-weighted forgetting. Our prior work [20] presented a trust-modulated retention rate, $\lambda_{\text{eff}} = \lambda(1 + \kappa(1 - \tau))$, as delivered. The formula is implemented, and it is called for every fact on the retention path. It has never had an effect. Its enclosing scheduler asks the schema a question before performing the per-fact trust lookup—does the trust table exist, and does the facts table carry an author column—and falls back to $\tau = 1.0$ when the answer is no. With

$\tau = 1.0$ the expression reduces to $\lambda_{\text{eff}} = \lambda$: not merely disabled, but *arithmetically identical* to a system that never had the feature.

The falling back is correct behaviour; it is what keeps an older store openable. What makes this worth reporting is where the missing data turned out to be. The author column is absent from the table the guard tests, and present—populated on every one of **4,340** rows—on a neighbouring provenance table that a different consumer in the same package already reads successfully. The same data, two consumers, one finds it and one does not.

That is the finding, and the obvious remedy is smaller than it looks. Those 4,340 rows cover **3,766 of 5,392 facts, or 69.8%**: re-keying the join would make the path effective for roughly seven facts in ten and leave the rest on the same fallback. Of the authors recorded, **3,997** are opaque capability digests and **237** are literally **unknown**, leaving 106 that name a readable principal. So this is a partial repair and a backfill decision, not a free fix, and the identity it would restore is machine-scoped rather than person-scoped.

We report that distinction because our own check got it wrong first. The join-liveness invariant of [Section 7.4](#) initially reported the populated row count and concluded “no migration and no backfill”—true about the column, misleading about the join. It now measures coverage over the guarded table and says so. A diagnostic that overstates its own remedy is an instance of the class it exists to detect, which is not a comfortable thing to publish and is the reason to publish it.

7.3 Why the learners’ own metrics cannot see this

All three failures are invisible from inside the mechanism, for the same reason in each case: the signal that would reveal the failure is derived from the mechanism that failed.

A stalled bandit still records plays, still advances its timestamps, and still reports a posterior. Its play count is not merely uninformative about learning; it is the thing that keeps rising while the posterior concentrates on a preference it never acquired. A guard that never passes raises nothing, appears in coverage, and is reachable by any call-graph trace. And a neutral fallback is the worst possible default for observability, because it composes into an identity: a decay multiplier that falls back to 1 and a reward that falls back to $\frac{1}{2}$ both produce output indistinguishable from correct operation.

This generalises past this system. *Implemented*, *reachable* and *effective* are three different questions. A grep answers the first. A call-graph trace answers the second. Neither answers the third, and only querying the store against real data does.

7.4 Two invariants that answer the third question

Both are read-only and on no hot path, and both are released in 4.1.5, reachable from the command line as a diagnostics subcommand. They are *not* in the 4.1.3 artifact whose measurements this paper reports—that release predates them—so the version of record for the invariants is one release later than the version of record for the measurements. We would rather state that than blur it: the numbers in [Section 7.2](#) were taken with the checks run against 4.1.3 stores from a working tree, and anyone re-running them today does so with 4.1.5 installed. They are deliberately mechanical: each is a few lines, each returns a verdict rather than a score, and neither requires a model.

Prior distance. For each Bayesian learner, assert that the posterior has moved away from its prior *mean* after n observations. The check reports STALLED when [Equation \(1\)](#) holds on every unit, MOVING when any unit has diverged, and INSUFFICIENT_DATA below an observation floor—because a cold learner sitting at its prior is correct, and a diagnostic that fires on correct behaviour is how a real warning stops being read. Motion off the prior is necessary for learning,

not sufficient; the check distinguishes *receiving signal* from *receiving nothing*, which is precisely the distinction the learner’s own counters cannot make.

What the prior-distance check gets wrong. Disclosing a diagnostic’s boundaries is part of shipping one, and this one has three.

It reports STALLED on *symmetric* signal, not only on absent signal: a learner receiving genuinely balanced outcomes—an even split of good and bad per arm, which a 50/50 comparison produces—satisfies Equation (1) and is reported as receiving nothing. The check separates *a posterior that has not moved* from one that has, which is not the same as separating signal from silence, and an operator seeing STALLED must still ask which they have. Second, the aggregate verdict loses per-arm detail in the direction that matters: it returns MOVING if *any* unit has diverged, so 164 stalled arms beside one live arm reports as moving. Third, the update applies a ceiling ($\text{MIN}(\text{cap}, \text{alpha} + \text{r})$), and an arm pinned at that ceiling no longer satisfies the identity even if every reward it ever received was neutral—a false negative. A fourth mode was found by this paper’s own second-round review: an observation floor applied in aggregate lets a store with many units and few observations report STALLED on units that were simply never played, so the floor now binds per unit as well. A fifth is the one the name invites: the check compares each unit’s posterior *mean* against the prior mean, to within 10^{-9} . It does not compare distributions. Two units at the same mean and different observation counts are different posteriors and will be sampled differently, and this check cannot tell them apart. “Prior distance” is therefore a convenient name for a narrower test than it sounds like, and the verdict it earns is *no learned preference*, not *unchanged*. Its registry is also explicit rather than exhaustive: it inspects the two Beta learners named in it, not every table in the store that might be learning something.

Join liveness. For each schema-guarded path, assert that the guard’s requirements are present in this store, and when one is not, search the remaining tables for the required column and report the row count found. The verdicts are LIVE, DEAD, and—the one that matters—SATISFIED_ELSEWHERE. The evidence is asymmetric and the check should be read that way: a satisfied guard means the path is *reachable*, not that it has ever run, whereas an absent requirement on a store that has been in service does establish that the path cannot have run against it. The findings rest only on the second direction. A dead path reported alongside the populated table that already holds its data is a one-line fix with no migration and no backfill; a dead path reported alone is a bug report. The check also records what the feature computes when the guard fails, because “switched off” and “computing exactly what being switched off would compute” are different facts for an operator.

7.5 What they found

Pointed at the authors’ own store, with no configuration and no hints:

Table 3: Both invariants, first run, against a live store.

Check	Verdict	Evidence
prior distance, channel bandit	STALLED	165/165 units, 1,405 obs
prior distance, source trust	STALLED	37/37 units, 629 obs
join liveness, trust forgetting	SATISFIED_ELSEWHERE	4,340 rows, 69.8% coverage

The third row is the result we did not anticipate. The check was written to report that a guarded path had never run; it also identified the table holding the data, the number of populated rows, and the absence of any migration requirement. The remedy was produced by the diagnostic, not by the engineer reading it.

7.6 From verdict to named cause to repair, and a test of the repair

A diagnostic earns its place by what follows it firing. The STALLED verdict prompted an investigation of the learning path, that investigation found defects, and two of them were repaired in public releases. We want to state the strength of that chain accurately, because it is weaker than a clean detect-to-repair story and still worth reporting.

What we can show: a shipped check returned a concrete verdict on a real store; work followed it; the defects that work found are named below and their fixes are installable. What we *cannot* show is a controlled attribution. The frozen state of [Section 7.2](#) covers 459 arms over 5,657 plays whose last activity is 2026-07-27, and this paper attributes that state to ledger writing being disabled at its only caller. The session-identifier defect below was measured later on a different population. The post-repair reading later still covers 165 arms. These are three overlapping observations of one subsystem at three times, not one population followed through a repair. We ran no replay and no single-factor ablation, so we cannot say which defect produced which part of the historical state, and a reader should not take the sequence below as that claim.

One defect the investigation found. The bandit records a play and separately writes a ledger row that a later authenticated outcome resolves. Outcome matching keys on the conversation a recall belonged to. The component that mints session identifiers and the component that matches them did not agree on which names counted: recalls arriving through an agent front-end’s subprocess were filed under identifiers the matcher deliberately excludes, because those identifiers are the placeholders a surface invents when no caller supplies one, and a placeholder must never be mistaken for a real conversation. Both halves were correct in isolation. Their intersection was empty, so no engagement signal could attach to any play.

The shape is what interests us, and it does generalise: **two components can each enforce a correct rule and still compose into a mechanism that cannot fire.** Neither component is wrong, so neither has a failing test, and the composition is what no unit test and no coverage report examines. We report the shape as an observation from one system, not as a measured prevalence.

Why a neutral default made it worse. A play that ended with no evidence was settled at the neutral reward rather than left unsettled. Under $\alpha += r$, $\beta += (1 - r)$, a reward of exactly $\frac{1}{2}$ adds $\frac{1}{2}$ to each parameter: the posterior mean does not move and the posterior *variance falls*. Repeated neutral settlement therefore does not leave a learner undisturbed—it makes the learner steadily more confident that its arms are average, and *quantifiably* harder for genuine evidence to move. From Beta(1,1), one genuine reward of 1 shifts the posterior mean by 0.167; after two hundred neutral settlements the same reward shifts it by 0.0025. That is a **67-fold** loss of responsiveness to real evidence, accumulated while the mean never moved and every counter the subsystem keeps continued to rise—which is why no threshold on those counters could have caught it. **Neutral is a commitment, not an abstention.** A learning system with no way to express *no evidence* will convert its own silence into confidence, and will do so while every counter it keeps continues to rise.

The repairs. Three changes, public in 4.1.6 and 4.1.7, with a fourth withdrawn in 4.1.8 and discussed below. The host variable the front-end actually sets is now among the names the resolver checks. Every synthetic identifier is now minted by one function and registered as synthetic, so the matcher recognises the ones that were previously minted ad hoc and silently accepted as real. And a play that observed nothing is closed as UNOBSERVED with its posterior left untouched. A fourth change turned adaptive ranking on by default, where it had sat behind an unset variable that kept the selector out of the call path entirely on a default install. **That fourth change was reverted one release later**, and the reason is the mechanism this section

has just described. With the selector live and most arms holding no settled reward, the weighting sampled for a query came from posteriors that had observed nothing, so two identical questions could weigh semantic against lexical evidence differently for no reason drawn from anything observed. The release notes withdrawing the default say exactly that. We record it because it is the same finding arriving from the opposite direction: the paper argues that an unsettled posterior is not a neutral one, and a shipped default had to be withdrawn because it is not. Adaptive ranking is opt-in again, and turning it on before the loop supplies real rewards trades determinism for nothing.

The synthetic namespace still exists, and should: an identifier a surface invented is not a conversation, and the matcher is right to refuse it. What was wrong was that some inventions were not declared as such.

What changed, measured. Read against the authors’ own store at 2026-08-25T01:23Z under 4.1.7: of **60** recorded plays, **40** closed UNOBSERVED with posteriors untouched—a state the earlier build could not produce, because it had no way to decline—**12** settled from a reported outcome, and 8 remained open. The mechanism that guaranteed neutrality is gone, and the abstention that replaced it is observable in the ledger.

The controlled result. Field state cannot separate “the repair worked” from “nobody used it”. A controlled ablation can, and this one changes exactly one factor: the identifier namespace a recall is filed under. Everything else—workload, corpus, simulated engagement, seed, and every production module in the path (`ContextualBandit`, the `is_conversation` predicate the hot path applies, `record_recall`, the engagement feature extractor, the reward model and the settlement pass)—is held identical. [Table 4](#) reports three arms over 120 recalls each.

Table 4: exp12, closed-loop ablation. One varied factor: the session identifier a recall is filed under. SLM 4.0 v4.1.9, seeded; repeated runs give identical verdicts and an identical maximum shift, with the instantiated-arm count varying 73–82 because arm selection is stochastic.

Arm	Tickets	Settled	Arms moved	max $ \bar{\theta} - \frac{1}{2} $
Defect: synthetic id	0	0	0 of 0	0.000
Repaired: conversation id	120	120	82 of 82	0.343
Control: no engagement	120	0	0 of 0	0.000

With the defect present, no outcome ticket is written, nothing settles, and no posterior moves. With it absent and the agent acting on what it was shown, every settlement is drawn from observed engagement (`ARTIFACT_OVERLAP`) and every instantiated arm moves off its prior mean, the largest by 0.343. **The loop closes, and the defect is what was holding it open.**

The third arm is the one that makes the second mean anything. It writes all 120 outcome tickets and differs only in that the agent then acts on something unrelated. Nothing settles and no posterior moves. A loop that reported learning there would be manufacturing reward from the existence of a ticket rather than from evidence, which is the failure this paper spent nine pages describing, and it would have been easy to ship and easy to mistake for success.

The same repair, observed in deployment. An ablation shows a mechanism can work. It does not show that the shipped fix reaches a real installation. The authors’ own store answers that directly, because the quantity the repair governs is observable in it: which namespace a recall’s session identifier came from. Before the repair, on 2026-08-23, 188 of 188 recalls were filed under synthetic identifiers that the matcher is designed to refuse. On 2026-08-24, the day the repair shipped, 142 synthetic and 9 conversation identifiers—the changeover, mid-day. On 2026-08-25, **0 synthetic and 7 conversation**. A cross-tabulation of the two tables the join depends on

shows the earlier state exactly: **zero** session identifiers appear in both `pending_outcomes` and the tool-event log across the whole history, which is why no engagement signal had ever attached to a play. The defect and its repair are both visible in the same column, and neither is inferred.

A fourth instance, found after this draft was frozen. Applying the same question to the other settlement path found the class again. The reward label is computed as 0.5 plus bonuses minus penalties, so an outcome carrying no engagement signal evaluates to exactly one half—and that value was persisted, becoming a mid-strength positive training label downstream and a Beta update that tightens a posterior around its prior. On the authors’ store, **492 of 492** recorded outcome scores were exactly this value, so not one of them reflected an observation. The batch settlement pass already declined to score an unsignalled row; the single-outcome path did not, and the two had disagreed since the mechanism shipped. Both abstain as of 4.1.9. We report this as an eleventh instance and deliberately do *not* renumber the ten of [Section 11.4](#): that count was frozen when the taxonomy was written, and the fact that the method kept finding instances afterwards is better evidence for the method than a larger number would be. It is also the sharpest illustration of the shape, because here the neutral value did not merely fail to inform the learner—it trained a failure as a moderate success.

What we still do not claim. A controlled ablation is not a deployment result. It shows the mechanism can transport a signal from an agent’s behaviour to a posterior, under engagement we generated; it does not show that the preferences a real deployment would learn are ones an operator would endorse, and it does not measure retrieval quality. All **165** arms in the authors’ live store remain at their prior. The twelve settled plays each carried a reported reward of exactly $\frac{1}{2}$, so they moved no mean. *The barrier to learning is identified, named and removed; the learning is not demonstrated.* Posterior movement requires sustained real engagement that a single-operator store over days does not supply, and we would rather publish that distinction than a curve. What this section does establish is narrower and, we think, more useful: an invariant that asks whether a mechanism *takes effect* located a defect that 459 healthy-looking arms, 5,657 recorded plays, and every counter in the subsystem reported as normal operation—and the repair it led to is in a public release a reader can install and inspect.

7.7 What does hold

Two admission boundaries on this path are enforced, and it is worth separating them from the three that are not.

Completion is decided by an independent gate. The bounded-loop engine terminates on a verdict from a checker the agent cannot edit or grade. The agent’s own assertion of completion is recorded in the ledger and is *never read* when deciding to terminate—the field appears only in assignments, never in a conditional. Each run leaves an append-only hash-chained ledger that can be re-verified after the fact, which makes it a fourth verifiable artifact alongside the completion manifest, the erasure receipt, and the audit chain. It is not part of the write transaction and does not participate in the completion manifest; execution verification and write verification are separate chains.

Unverifiable evidence is admitted for display and barred from learning. External gate-verified execution receipts enter through a versioned contract. Where the producing tool cannot itself prove the receipt, the record is persisted with an explicit trust level and an `eligible_for_learning` flag set false: it may be displayed and observed, and it cannot promote a memory, alter learning, or assert execution authority. The tier is set by the *verifiability* of the evidence rather than by its content or the reputation of its source. The package deliberately

neither imports the producing tool nor reimplements its log grammar; the tool’s own command is the versioned protocol port. At the time of writing this boundary has been exercised once, by a demonstration run, which is the honest extent of the claim.

7.8 What is not governed

- The three mechanisms in [Section 7.2](#) were instrumented and inert at the version whose measurements this paper reports. Nothing here should be read as a claim that any of them works, before or after the repairs of [Section 7.6](#).
- The skill-evolution pipeline is implemented, gated, and has never executed. Parent-version lineage and generation tracking are not reported.
- The invariants in [Section 7.4](#) are *admission* claims, not outcome claims. We present no evidence that gated learning produces better agent behaviour, because we have run no such experiment. A reader who takes this section as a retrieval-quality or task-success result has been misled.
- The loop ledger is verifiable but is not bound into the completion manifest.
- **Two of the ten shapes have a mechanical detector; eight do not.** The invariants cover Bayesian learners that have not moved and schema-guarded paths whose requirements are absent. A liveness probe reading its own output, an availability check on a re-violable invariant, a distribution channel forking its artifact, and a metric differencing two arms whose costs were never comparable are each named in [Section 11.4](#) and each still require a human to notice. Mechanising more of them is the obvious continuation and we make no claim to have done it.
- Two of the three mechanisms have had their named cause repaired in 4.1.6–4.1.9 ([Section 7.6](#)); *repaired* here means the barrier is removed, not that either learner has been shown to learn. The third—re-keying the trust join onto the provenance table—is scoped work we have not done, and [Section 14](#) says so rather than implying otherwise.

8 System Model and Verified Design Invariants

The governing invariant states, as a design goal, that a memory operation yields one authenticated actor, one profile generation, one policy decision, one durable receipt, and one verifiable completion state across the registered projection owners. V4 realises this full envelope on the canonical HTTP `/remember` path and internal ingestion; declared MCP tools and selected CLI mutations obtain a shared policy decision without the full envelope. We label the five scoped correctness properties below *Verified Design Invariants* (VDIs). This section is an assurance case, not a proof development, and the distinction is deliberate rather than apologetic. The definitions are formal: they fix the state a claim quantifies over, so that a reader can say precisely what would falsify it. The invariants are not deduced from axioms; each is a property of the canonical write/read path established by an implementation trace and measured directly. We chose that instrument because the claims here are about what a running system does to a physical store, and a proof about a model of that system would establish a different thing. Each justification traces the shipped code, and measured evidence for each invariant appears in [Section 11](#). A reader wanting deduced theorems should read this section as the falsification conditions for the evaluation that follows, not as a substitute for it.

Definition 8.1 (Actor context). *An actor context is the frozen tuple $A = (p, R, P_{\text{allow}}, \pi, g, \Sigma, \delta, \tau, h)$ where p is the server-derived principal (never from the request body), R is the frozen role set, P_{allow} is the profile allowlist, π is the active profile ID, g is the active generation, Σ is the scope allowlist, δ is the transport channel, τ is the resolved client host,*

and h is the SHA-256 prefix of the session token (first 16 hex chars; no raw session material retained). A is authenticated iff $p \neq \varepsilon$ and $ANONYMOUS \notin R$.

Definition 8.2 (Projection obligation and owner). A projection obligation records $O = (\text{op}, \rho, \omega, s, v, c)$ where ρ is the owner name, $\omega \in \{\text{APPLY}, \text{ERASE}\}$, $s \in \{\text{PENDING}, \text{APPLIED}, \text{VERIFIED}, \text{FAILED}, \text{COMPENSATED}, \text{ERASED}\}$, and c is an owner-specific SHA-256 checksum. Terminal successes: $\{\text{VERIFIED}, \text{ERASED}\}$. The three registered admission owners are $\Omega = \{\text{bm25}, \text{temporal}, \text{vector}\}$.

Definition 8.3 (Completion manifest). A completion manifest is the frozen record $M = (\text{op}, \pi, \sigma, b, n, \mathcal{E}, h_M, t_c, t_u)$ where $\sigma \in \{\text{COMPLETE}, \text{DEGRADED}, \text{FAILED}\}$, b is the `all_met` flag, $\mathcal{E}$ is the sorted tuple of *OwnerEvidence* dicts, and h_M is the envelope hash: on M037-capable databases an installation-key HMAC-SHA256 (version 2, refusing a version-1 downgrade); on older schemas, an unkeyed SHA-256 self-consistency check.

Verified Design Invariant 8.1 (Bounded In-Process Generation-Epoch Rejection). **Claim.** If the binding epoch increments from g_0 to $g_1 > g_0$ after an admission event (k, π, g_0, t_0) is recorded but before `_handle_admission` executes, the handler raises `ValueError("epoch is stale")` before any projection write occurs. **Scope:** in-process, single-writer, 300-second TTL, not CAS, not SQLite-backed, not persistent across restarts.

Implementation trace. Under `_binding_lock`, `admitted_epoch` returns $g_0 \neq g_1$, so the guard fires before `command_impl.submit()` is reached; conflict epoch $-1 \neq g_1 \geq 0$ likewise fires. Measured: exp7, 199/200 (Section 11); the single failure was a transient writer unavailability on the positive control, not a fence misjudgement, and it is analysed in Section 11.1.

Verified Design Invariant 8.2 (Completion-Manifest Integrity and Honest State). **Claim.** (i) On M037-capable databases the manifest envelope is sealed with an installation-key HMAC-SHA256 (version 2, refusing a version-1 downgrade at verification): a DB-only writer without the installation key cannot forge a seal that `verify_manifest` accepts (authenticity, not merely corruption detection). On older schemas the seal is an unkeyed SHA-256 that detects accidental corruption (bit flips, partial writes) but is not tamper-evidence against a malicious writer. (ii) `all_met = True` iff every obligation is in terminal-success state; a DEGRADED manifest is never reported as success.

Scope: authenticity holds under installation-key secrecy on M037; legacy unkeyed v1 is corruption-detection only; an attacker holding the installation key is outside the model.

Implementation trace. `verify_manifest` recomputes the versioned seal over the canonical JSON fields and refuses a v1 downgrade on M037; HMAC unforgeability (resp. SHA-256 collision resistance) yields an invalid/different digest on any single-field mutation. `derive_state` maps `all_met=True` iff all obligations are in $\{\text{VERIFIED}, \text{ERASED}\}$, any other state routing to DEGRADED. Measured: exp2, 200/200.

Verified Design Invariant 8.3 (Covered HTTP Read Profile Isolation). **Claim.** On covered HTTP read paths, an actor lacking READ permission for the active profile π_A is rejected before any query executes; an authorized actor cannot retrieve private rows (`profile_id = \pi_B \neq \pi_A`).

Scope: authorization-layer isolation via SQL predicates and RBAC on covered HTTP paths; MCP stdio and CLI direct-call paths are not covered by this gate; direct SQLite file access is outside the threat model.

Implementation trace. Two independent controls. (a) Storage predicate—covered read paths parameterize every query with the actor’s `profile_id` from the server-side *ActorContext*, so a query for π_A never returns π_B rows; measured by exp5 with a positive control (200/200) on two read paths, `get_facts_by_ids` and `TemporalChannel`. (b) HTTP RBAC middleware—the read gate rejects unauthorized actors with 401/403 before a query runs on enumerated sensitive routes; this is established by code inspection of the middleware, not by exp5. An end-to-end unauthorized/authorized HTTP probe is future work.

Verified Design Invariant 8.4 (Erasure Completeness over Registered Owners). ***Claim (conditional).** If the returned receipt has `all_erased=True`, then at `finalize()` time each owner $o \in \Omega$ returned `erased=True` with empty residue. Partial residue sets `all_erased=False`; the receipt is honest about the gap.*

***Scope:** limited to the three registered owners (`bm25`, `temporal`, `vector`) present in `ErasureService._owners` at erasure time; projection stores not wired as registered owners are not covered.*

***Implementation trace.** `finalize()` calls `prove_erased()` per owner—a live physical re-query—and seals over all fields with HMAC-SHA256 (v2 on M037); refusing a v1 downgrade makes the receipt unforgeable by a DB-only writer without the installation key. Measured: exp1, 200/200.*

Verified Design Invariant 8.5 (As-Of Temporal Demotion and Retention Non-Increase).

***Claim (a).** A system-invalidated fact ($t_{si} < +\infty$) receives score factor 0.25; after per-channel re-sort it ranks strictly below any fact with the same positive raw score and demotion factor 1.0.*

***Claim (b).** The production Ebbinghaus function $R(t) = e^{-t/S}$ in `ebbinghaus.py` is non-increasing in $t \geq 0$ for every $S \geq S_{\min} > 0$.*

***Scope:** with `as_of = q`, supersession demotes only when it is visible at q (`system_expired_at ≤ q`), and half-open event-validity (`valid_from > q` or `valid_until ≤ q`) applies a separate 0.5 factor. Demoted facts remain present and re-ranked—`as_of` is point-in-time demotion, not snapshot isolation—and the filter fails open on error.*

***Implementation trace.** (a) `temporal_validity_filter.py` multiplies system-invalidated scores by 0.25 and re-sorts descending; $0.25s < 1.0s$ for any $s > 0$. (b) `math.exp(-t/S)` is non-increasing in t ; IEEE 754 underflow eventually clamps to 0.0, making R non-strictly constant there. Measured: exp6a, 200/200.*

9 Threat Model and Capability Coverage

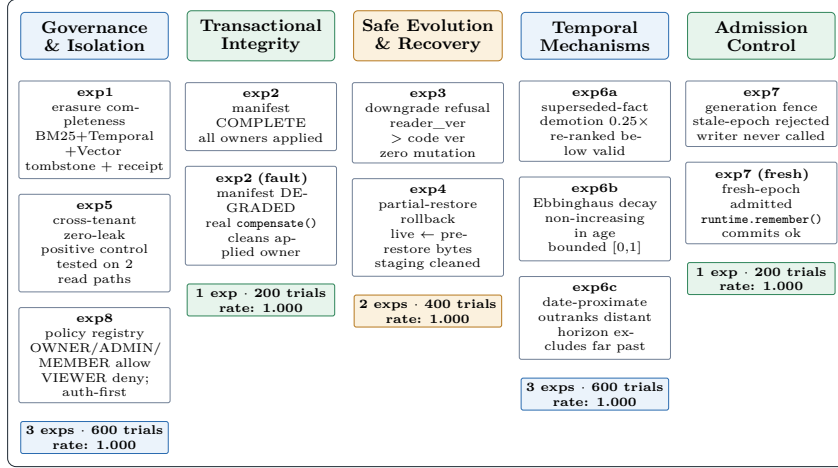
SLM 4.0’s threat model is bounded to adversaries operating above the OS boundary or within the runtime’s own failure modes. Isolation is an authorization-layer control; a single OS user with direct access to the SQLite store files is outside this model.

T1 — Cross-tenant read. Read paths apply a canonical scope predicate enforcing `profile_id` equality and the personal/shared/global scope algebra at query time; cross-scope access requires explicit shared membership (recorded at write time) or the `global` scope flag, default-deny. The all-channel statement rests on call-path inspection and per-channel unit tests; *measured evidence* is exp5 (200/200 repeated executions), which exercises the scope predicate on two paths (`get_facts_by_ids` and the temporal channel) with a positive control—it is not an empirical proof for every channel.

T2 — Replayed or stale-epoch write. The generation fence records the admitted epoch at ingestion; a mismatch raises `ValueError("epoch is stale")` before any projection writer is called. *Evidence:* exp7 (199/200 repeated executions; see [Section 11.1](#)). *Scope:* in-process, 300-second TTL, on the canonical remember path only.

T3 — Projection inconsistency. Each canonical write registers an obligation for the three registered owners and records a `CompletionManifest` sealed with an installation-key HMAC-SHA256 (v2 on M037). If any owner fails to verify, the manifest carries an honest DEGRADED status; the maintenance loop redrives pending obligations. DEGRADED is never silently discarded or promoted to COMPLETE. *Evidence:* exp2 (200/200 repeated executions).

SLM 4.0 Reliability Evaluation: 10 Deterministic Property Scenarios



All 10 scoped properties held in 200 repeated executions each; repetitions are flakiness checks, not independent statistical samples

Figure 12: Failure-class taxonomy from Section 11: the ten measured guarantees grouped into five categories that map to SLM 4.0’s five technical areas. *Governance & Isolation* covers projection-store erasure completeness (exp1), cross-tenant zero-leak with positive control (exp5), and operation policy correctness with authentication-before-role-check (exp8). *Transactional Integrity* covers completion manifest correctness and real `compensate()` behavior (exp2). *Safe Evolution & Recovery* covers migration downgrade refusal (exp3) and backup-restore atomic rollback (exp4). *Temporal Mechanisms* covers superseded-fact demotion (exp6a), Ebbinghaus decay monotonicity (exp6b), and time-window inference (exp6c). *Admission Control* covers generation-fenced admission rejection and fresh-epoch commit (exp7). Each scenario ran 200 deterministic repetitions; every scoped property held.

T4 — Memory-content secret leakage. The shared queryable-write chokepoint applies `scrub_secrets_for_ingest` unconditionally before canonical and projection persistence. *Disclosure*: mesh coordination messages are not subject to the full admission scrub (fail-open); sender-side filtering remains required for sensitive payloads.

T5 — Migration downgrade or backup corruption. Two measured controls: a newer-stamped database is refused on the deferred migration pass with the table-set preserved (*measured*: exp3, 200/200), and a partial-restore failure rolls live data back to pre-restore bytes (*measured*: exp4, 200/200). The Scale Engine’s projection parity-verify-before-promote and rollback path is established by code inspection; a fault-injection experiment driving prepare/verify/promote/rollback is future work.

T6 — Unauthorized skill mutation. Skill evolution is gated by a per-cycle budget and a blind-verified pipeline to quarantine. The MCP entrypoint is gated by `OperationPolicyRegistry` (EVOLVE_SKILL, roles {OWNER,ADMIN}). *Residual gap*: the MCP actor is synthesized from deployment mode with no live session principal, so no human-approval step is interposed; `auto_approve` is a configuration gate, not a live-RBAC control.

T7 — Mesh relay content exposure. Mesh messages are redaction-scrubbed on receipt but not admission-scrubbed (fail-open). Cross-machine coordination uses leaderless LWW—not consensus, quorum, or a replicated database—with TLS/pinning and HMAC transport available and ordinary storage writes not invoking the fence primitive.

Deferred controls (D1–D3). Per-device mesh key revocation (future work); cryptographic multi-tenancy (delegated to OS-level disk encryption); inference-time injection gating (complementary to MemGate [31], layerable above the retrieval surface).

10 Implementation

SLM 4.0 spans more than 25 subsystems (`access`, `compliance`, `core`, `mesh`, `retrieval`, `storage`, `temporal` components in `encoding/retrieval`, `optimize`, `code_graph`, `learning`, `trust`, and others), exercised by a release gate whose exact collected, selected, passed, skipped, and deselected counts are bound to the final source SHA, command, and result artifact in the release manifest. The exact commands and environment are documented with the released harness.

We separate two kinds of evidence here, because the bundle accompanying this paper does not treat them alike. The eleven experiments of Section 11 ship with complete per-experiment result artifacts, and every number this paper quotes from them is reconstructable from those files. The repository-wide suite logs in the same bundle are *not* of that standard: one is truncated before its terminal summary and two end on failing tests, all of which are the concurrency-contention artifacts described in Section 12. We include them because omitting them would be worse, and we state plainly that they substantiate the experiment results and not the repository-wide gate wording above. A reader should treat the gate as a release process we describe and the experiments as the evidence we offer.

The system runs in three privacy modes (A, B, C as described in Section 4) and is delivered through a CLI (`slm` with agent-native `-json`), an MCP server (HTTP and stdio, with configurable tool profiles from `core/14` tools to `whole/all` registered), a dashboard, a Python SDK, editor and proxy integrations, and **nine separately packaged framework adapters**. The adapters are packaged as standalone distributions outside the base wheel; in this release the repository’s adapter package metadata has been updated to admit SLM 4.0, and the small core surface they depend on (`SLMConfig`, `MemoryEngine`, `Mode`) is present and load-verified against 4.1.3. Published adapter releases and full per-framework integration testing are pending.

The new architectural modules for the transaction spine, policy registry, admission gateway, schema-capability record, backup coordinator, and erasure manifest are additive: they sit beneath the existing product, and surfaces adopt them incrementally behind transport tests.

11 Reliability Evaluation

We evaluate SLM 4.0’s durability, isolation, temporal, and admission guarantees by direct measurement against an installed package reporting version 4.1.3 on Python 3.14.5—a fault-injection and mechanism evaluation, not an external conversational-accuracy benchmark.

Method. Each experiment imports the installed `superlocalmemory` package and drives production code paths—real SQLite databases built through the production initialization sequence, the real backup coordinator, the real migration runner, the real authorization/scope layer, the real temporal machinery, the real `MemoryTransactionService`, the real `ErasureService` with concrete projection owners from `concrete_owners.py`, the real `generation_fence` module, and the real `_DEFAULT_REGISTRY`. Synthetic inputs are limited to a deterministic vector where a vector is structurally required, plus the synthetic projection owners of exp2 (superseded by the real owners in exp2b; see the per-experiment disclosures below).

Every experiment is **bracketed**: it fails if the mechanism does nothing *or* does the wrong thing. Isolation (exp5) carries an explicit **positive control**—the requester must still see its own data through the same read paths. The harness is fail-loud: an infrastructure error crashes the run. Per-experiment setup disclosures: exp2 uses synthetic `_TrackingOwner` stand-ins; exp7 builds the full production schema through the migration runner’s `apply_all` over both stores,

rather than an enumerated migration subset; exp8 is CPU-only with no SQLite database. Among the rest, only exp1 exercises the full production projection owner set.

11.1 Results

Ten of the eleven scoped component properties held in every one of their 200 deterministic repetitions (Table 5) at the disclosed scope of the table. The eleventh—the generation fence—held **199 of 200**. The released evidence bundle therefore reports **2,199/2,200**, where the previous version of this paper reported 2,200/2,200 on the same eleven scenarios. The real-owner replication of Section 11.1 (exp2b, 200 repetitions) is included in that total, and every experiment is executed by the single runner `benchmark/run_all.py`.

We report the single failure rather than re-running until it disappears, and it is worth being explicit about why: selecting a favourable run is the precise practice this paper’s evidence standard exists to forbid, and we have criticised it in our own prior work elsewhere in this section.

The failing repetition was the *positive* control—a fresh-epoch write that should have been admitted—not the stale-epoch rejection. It failed because the canonical writer was transiently unavailable, so the fence’s judgement is not implicated. Diagnosing it produced a further instance of the failure class in Section 11.4. The writer collapsed three unrelated exception types into one message: an unavailable admission journal (contention, retryable), lost writer ownership (another process holds the lease), and a write-coordinator error—which is *the same type the generation fence raises to reject a stale epoch*. A spurious fence rejection and a disk stall therefore produced identical text, for the operator reading a log and for this experiment recording a failure. The three causes are now named in the message, so the next occurrence is attributable. The underlying transient is unresolved. Re-running the same scenario five further times produced 200/200 on each, so the observed rate is one occurrence in 1,200 repetitions rather than one in 200; a rate that low is characterised, not diagnosed, and it is listed in Section 12. No single scenario exercises the full end-to-end path (HTTP auth → `ActorContext` → journal → fence → owners → obligation ledger → manifest → ANN); transport, multi-process, mesh, and long-lived deployment fault-injection are future work.

These are deterministic property scenarios, so the repetitions test for flakiness and environment sensitivity rather than sampling a defined random population. We therefore report no confidence interval or inferred failure probability.

Experiments exp7, exp2, exp5, exp1, and exp6a correspond to the five Verified Design Invariants of Section 8 (Verified Design Invariants 8.1 to 8.5)—generation fence, completion manifest, read isolation, erasure completeness, and as-of demotion—covering 1,000 of the 2,200 released repetitions; the remainder cover real-owner validation (exp2b), policy-registry evaluation (exp8), and further correctness properties (migration downgrade, backup rollback, and additional temporal mechanisms).

Real-owner validation (exp2b). Because exp2’s manifest/compensate check uses synthetic owner stand-ins, we additionally ran **exp2b** with the real `Bm25Owner`, `TemporalOwner`, and `VectorOwner` driven through `MemoryTransactionService`: all 200 repetitions held—100 committed writes yielded a COMPLETE manifest with all three real projections physically present, and 100 with an injected Bm25-owner failure yielded a DEGRADED manifest in which the real `service.compensate()` removed the surviving temporal projection to zero residue and the failed owner left no partial rows. *Scope*: the Bm25 fault is an isolated owner-verify failure (the live retrieval-engine heal path is not exercised), and the `VectorOwner` runs in its `embedding_metadata` metadata mode with the `sqlite-vec` ANN index out of scope, as in exp1.

Table 5: Reliability evaluation—200 deterministic repetitions per scenario, SLM 4.0 v4.1.3 / Python 3.14.5 / macOS-26.4.1-arm64. Ten scenarios held 200/200; the generation fence held 199/200.

#	Guarantee (exact as measured)	Metric	Trials	Held	Rate
1	Bm25Owner + TemporalOwner + VectorOwner erase all wipe-tenant projection rows from <code>bm25_tokens</code> , <code>fact_temporal_validity</code> , and <code>embedding_metadata</code> ; tombstones and a verifiable receipt persisted; keep-tenant content-hash unchanged	complete-erasure	200	200	1.000
2	MemoryTransactionService: committed op → manifest COMPLETE with both owners applied; faulted op → manifest DEGRADED, successful owner’s projection removed by real <code>service.compensate()</code> , failed owner had no projection residue	manifest-correct	200	200	1.000
2b	Real Bm25Owner, TemporalOwner, and VectorOwner: committed writes yield COMPLETE manifests with all projections present; injected Bm25-owner failure yields a DEGRADED manifest and removes the surviving temporal projection to zero residue	real-owner-correct	200	200	1.000
3	Newer-stamped DB raised <code>SchemaVersionError</code> on the deferred pass with no change to the table-name set	refuse+preserve	200	200	1.000
4	Partial-restore failure rolls live data back to pre-restore bytes	rollback+clean	200	200	1.000
5	Personal rows of one tenant never leak to another on the two tested read paths (<code>get_facts_by_ids</code> and <code>TemporalChannel</code>), with positive control	zero-leak	200	200	1.000
6a	Superseded facts kept but demoted 0.25× and re-ranked below valid	correct-demotion	200	200	1.000
6b	Ebbinghaus retention non-increasing in age, bounded [0, 1]	monotonic-decay	200	200	1.000
6c	Date-proximate events outrank distant ones; horizon excludes far past	correct-window	200	200	1.000
7	Generation fence rejects a stale-epoch admission (<code>WriteCoordinatorError</code> , writer never called) and admits a fresh-epoch admission	fence-correct	200	199	0.995
8	<code>_DEFAULT_REGISTRY.evaluate()</code> : REMEMBER allowed for OWNER/ADMIN/MEMBER; VIEWER→REMEMBER denied; unknown kind fail-open in local, fail-closed in company; unauthenticated actor denied with <code>authentication_required</code> before role check	policy-correct	200	200	1.000
Execution total			2,200	2,199	0.9995

11.2 Real-Scale Performance

Version of record for this subsection. The figures below were produced against an installed package reporting version **4.0.0**—the version this paper presents—whereas the reliability evaluation of [Section 11.1](#) was re-run on 4.1.3. We state the split rather than blur it, and the reason for it is the point of this paper: the reliability harness was re-run because a later release *changed* its result, from 200 of 200 to 199 of 200 on one scenario, and a changed result must be reported at the version that produced it. Nothing in the release notes between 4.0.0 and 4.1.3 touches the read, write or embedding paths measured here, so we did not manufacture a version match we had no evidence for. A reader who wants these numbers at a later release should re-run the released harness rather than read a re-labelled table.

Performance was measured against a retained memory-store database copy (1,232 MiB; 768-d embeddings), driving the real engine with warm caches, GC disabled during timed loops, and embedder warm-up excluded from timing. *Provider scope:* these figures use the in-process `sentence-transformers` embedder (`nomic-ai/nomic-embed-text-v1.5`), whose cold model load is 10,009.9ms and is excluded; an earlier measurement of this suite against a warm Ollama endpoint recorded a 541.8ms warm-up. Embedding provider and database size both materially affect these numbers, so both are stated rather than left implicit; figures produced under different providers are not directly comparable.

Concurrency. [Figure 13](#) sweeps 1–16 GIL-shared threads alternating `store_fast` writes and direct `recall` (the legacy bypass path, not the governed write path). Throughput rises from 0.9 to 18.4 ops/s across 1–16 workers, while median per-operation latency degrades from 74.9ms to 1,009.4ms as GIL and write-lock contention grow, with **zero “database is locked” errors observed at any level** under the single-writer, WAL-journaled design; we report the observation without attributing multi-process or transport causality.

Latency. [Figure 14](#) reports percentiles over $n=300$ warm operations on the 1,232 MiB store. Recall is embedding-bound: 1.93s at the median and 4.38s at p99 with the in-process sentence-transformers provider. The 42.0ms median for the *legacy direct store_fast* write path on that store bypasses canonical admission, the policy registry, the encrypted journal, projection obligations, and the completion manifest, so it does *not* bound governed-path cost.

Governed write-envelope overhead. To bound the control-plane cost directly, we measured the canonical governed write envelope in-process (`CanonicalRememberRuntime.remember()`—the same path the HTTP handler calls after request parsing: encrypted admission-journal prepare/dispatch/commit, the single-writer `WriteCoordinator`, the canonical SQLite write, and the projection obligation-ledger insert) against the ungoverned in-process write on a fresh store, $n=1,000$ warm calls each after a 30-call warm-up burst. The previous version of this paper *reported* a governed envelope at p50/p95/p99 = 3.522/4.427/5.297 ms against 1.835/2.379/2.569 ms ungoverned, and derived from that pair a control-plane overhead of 1.687 ms (p50) to 2.728 ms (p99). Those five numbers are stated here in the past tense deliberately: they are the claim under retraction, not a finding of this version, and neither the pair nor the difference should be quoted from this paper as current.

We withdraw that overhead figure. Re-measuring it on the current release returned a *negative* p50 delta, which is impossible for a genuine overhead: a governance envelope can only add work. The number was not mis-measured, it was mis-defined—and our first account of *why* was also wrong, which is worth recording because a reader can check it.

We initially concluded that the two paths were not nested: that the comparand performed three writes in one transaction where the governed writer performed one. That is false. The governed path constructs the same `IngestionCommand` around the same writer callback and calls `submit()` on it, then adds the journal, the coordinator binding and the projection-obligation records. The comparand is a logical *subsequence* of the governed path, so the governed path does strictly more work—and still measures faster at the median. Structural nesting was never the problem.

We then offered a second explanation, and it is also wrong. We said that the governed path holds one transaction across a run of writes while the comparand commits once per call, so the subtraction measured commit batching. It does not. The write coordinator issues **BEGIN IMMEDIATE** and commits once for *each* queued write. Both real arms perform one transaction and one commit per write, and nothing is batched across calls in either.

We are not going to propose a third cause. Two explanations, each consistent with the code we had read at the time, each checkable, each wrong. We know of two ways the arms differ besides governance: the governed path reuses one long-lived connection where the comparand is called standalone, and the governed path sets SQLite’s **synchronous** pragma per write lane while the comparand never sets it at all. Either could produce the sign. We have not isolated which, and naming one would repeat the error this paragraph exists to record.

Two further facts belong here. Interleaving the arms under a recorded seed did not change the sign, so this is not an ordering artifact. And each arm writes to its own temporary database, so they never contend for the same file; sharing one would trade that for cross-arm interference. Neither arrangement isolates the envelope, and that is the point: no choice of comparand does, which is why [Section 11.2.1](#) stops looking for one.

The released result artifact carries our *first* explanation in its interpretation field, and we withdraw it there as well. Its measurements are unaffected: they are absolute per-arm costs, and the field it contradicts is a note, not a number. Its **overhead_well_defined** flag already reads false.

[Table 7](#) therefore reports the three paths as absolute costs and states no difference between them.

11.2.1 Measuring the envelope instead of subtracting pipelines

The failure above is not a measurement error, it is a method error: we asked what governance costs by differencing two programs, and no amount of care in choosing the comparand fixes a question posed that way. So we stopped subtracting. The envelope is a known set of components on a known path, and each can be timed where it runs, inside the same transaction on the same connection under the same pragmas as the work it is being compared against. Nothing is bypassed, no comparand is constructed, and there is consequently no comparability question left to get wrong. [Table 6](#) reports $n=400$ warm governed writes at 4.1.9.

Table 6: exp13, governance envelope cost by component, measured in place. $n=400$ warm calls, SLM 4.0 v4.1.9, in-process, temporary filesystem. Total governed write p50 = 11.00 ms.

Component	p50 (ms)	Share of write
Admission journal — commit (terminal row)	3.137	28.5%
Admission journal — prepare (encrypt + durable row)	3.109	28.3%
Admission journal — dispatch transition	1.480	13.5%
Projection-obligation ledger insert	0.042	0.4%
Generation fence (epoch check)	0.0019	0.02%
Envelope total	7.77	70.6%

The distribution is the result, and it is not the one we expected to report. The envelope is roughly seventy per cent of a governed write, and *essentially all of it is durability*. The two components that make the write *governed* rather than merely durable—the generation fence that decides whether an epoch may still write, and the obligation ledger that makes cross-store completion checkable—cost 0.044 ms together, four tenths of one per cent. The remaining 7.73 ms is three transitions of an encrypted write-ahead journal.

Governed memory is not expensive because it is governed. It is expensive because it is durable. The policy machinery is microseconds; the write-ahead record is milliseconds. That distinction matters to anyone deciding whether they can afford this design, because the two halves have entirely different engineering responses: a deployment that needs lower write latency should reach for journal batching, group commit or a faster device, and will find nothing worth removing in the governance decisions themselves. Repeated runs gave envelope shares of 70.3, 70.6, 70.7 and 71.3 per cent, with the fence between 0.0019 and 0.0021 ms in every one; the journal component tracks the device and ranged from 7.29 to 8.47 ms, which is the expected sensitivity and the reason the share is quoted rather than the absolute alone.

Scope. In-process; HTTP transport, request parsing and the trust hook are excluded. Databases sit on a temporary filesystem, so a persistent disk raises the journal components specifically. The sum of component medians is not the median of the sum, so the share is indicative rather than an identity.

Table 7: Write-path cost by arm, interleaved under a recorded seed, $n=1,000$ per arm. The arms are not comparable in ways we have not isolated, so no difference between them is a governance overhead. The retained artifact records the platform and the module measured but no package version, and we do not assign one here.

Path	p50 (ms)	p99 (ms)
Governed: journal, coordinator, writer	17.2	130.2
Ungoverned: <code>IngestionCommand.submit()</code> , own transaction per call	51.1	99.8
(<i>artifact</i>) bare writer, no caller transaction	173.7	343.5

The third row is included for completeness and is not a peer of the other two: the innermost writer is built to run inside a caller-managed transaction, and calling it standalone adds a per-call `fsync` that neither real path pays. It bounds nothing.

The governed path is the faster of the two real paths. We report that and stop there, because every account we have given of *why* has failed on inspection. What survives is the observation itself: adding a journal, a fence, an obligation ledger and a single-writer queue did not make this write path slower than calling its own inner command directly. That is worth knowing and it is not a measurement of governance overhead. The defensible statement about cost is the absolute one: a governed write completes in 17.2 ms at the median. That figure is measured on freshly created temporary stores, not on the 1,232 MiB copy used for the recall, concurrency and resident-memory results above; it is a floor, and a larger store will not be faster. *Scope:* in-process (HTTP transport and request parsing excluded; loopback HTTP typically adds 0.5–2 ms); databases on a temporary filesystem, so persistent-disk `fsync` on the first journal write may raise p50 toward the low tens of milliseconds—these are floor numbers—and the comparand arm is `IngestionCommand.submit()` called standalone. A full end-to-end HTTP benchmark under multi-agent concurrency remains future work.

Memory stability. Figure 15 tracks RSS across 60 s of sustained load. After initial warm-up, RSS remains within 436.4–440.7 MB with no monotonic upward trend (4 MB rounded spread; final 438.0 MB). The harness classifies this window as `NONE` for its bounded-spread leak indicator; a definitive leak verdict still requires a longer soak with an explicit idle-recovery phase (future work).

11.3 Retrieval Quality: Carried-Forward V3 LoCoMo Evidence

The experiments above measure durability, isolation, admission, and temporal *mechanisms*; they are not a retrieval-accuracy benchmark. For retrieval quality we carry forward the published LoCoMo [13] evidence for the V3 architecture that SLM 4.0 productionizes; we do *not* rerun it against the V4 release artifact, and every figure is protocol-scoped.

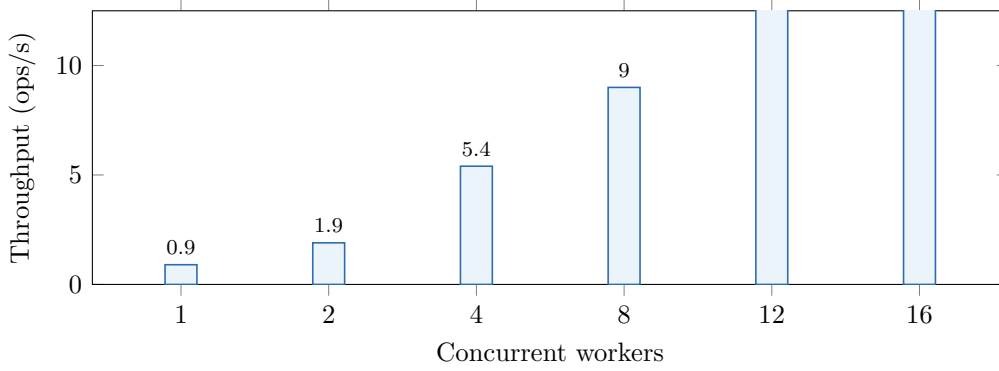


Figure 13: Mixed remember/recall throughput versus concurrency on a ≈ 1 GB store (Mode B, Ollama embeddings, `ThreadPoolExecutor`). Throughput scales from 1.0 to 18.4 ops/s across 1–16 workers while median per-operation latency rises from 74.9 ms to 1,009.4 ms (0.84–1.23 s), and **zero writer-lock errors** occur at any level: the single-writer WAL model serialises writers without contention failures.

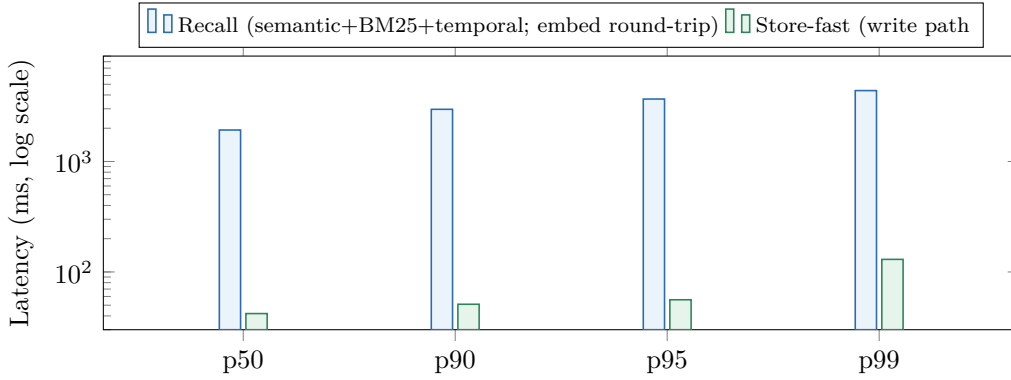


Figure 14: Operation-latency percentiles over $n=300$ warm operations on a 1,232 MB store (Mode B; log scale). Recall latency uses the in-process sentence-transformers provider (`nomic-ai/nomic-embed-text-v1.5`); the write path without embedding (store-fast) is 42.0 ms at the median. Latency is therefore a property of the configured embedding provider, not of the control plane; embedder warm-up (10,009.9 ms, in-process sentence-transformers) is excluded from timing and GC is disabled during each timed loop.

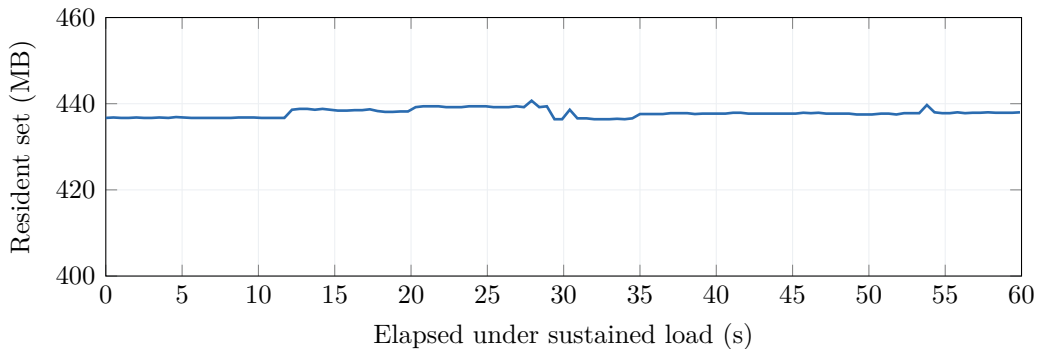


Figure 15: Process resident-set size over 60 s of sustained mixed load on a 1,232 MB store. RSS remains within 436.4–440.7 MB (4 MB rounded spread; final 438.0 MB) with no monotonic upward trend—consistent with a bounded working set (canonical pages plus embedding buffers) rather than an unbounded leak. A longer soak with an explicit idle-recovery phase is noted as future validation (Section 12).

Table 8: Carried-forward LoCoMo evidence for the V3 architecture (protocol-scoped; *not* a rerun V4-release benchmark). Source: [21].

Configuration	LoCoMo	Protocol scope
Mode A Raw	60.4%	10 conversations; 1,276 scored questions; local embeddings, local retrieval, zero-LLM answer construction.
Mode A Retrieval	74.8%	Same set; local retrieval followed by a single GPT-4.1-mini answer-formation pass—the standard LoCoMo answering protocol, distinct from the zero-LLM Raw row above. Published V3 result, not re-derived on the V4 artifact; a hash-pinned V4 re-run is future work.
Mode C	87.7%	Conv-30 only; 81 scored questions; text-embedding-3-large plus GPT-4.1-mini answer generation and judge. Not a full-dataset result.

The V3 Mode A Retrieval result breaks down by category as 72.0% single-hop, 70.3% multi-hop, 80.0% temporal, and 85.0% open-domain. An information-geometric ablation on six V3 LoCoMo conversations reported +12.7 pp under the published protocol; the displayed one-decimal endpoints (71.7% with versus 58.9% without the geometric layers) imply 12.8 points because of rounding. Both are V3-architecture results, not re-verified on V4; an end-to-end V4 comparative benchmark against LoCoMo and LongMemEval is preregistered future work.

11.4 A Failure Class Found by Operating the System

The harness in Section 11.1 did not find any of what follows. That is the first thing to say about it, because conflating the two would misrepresent both. The scenarios in Table 5 are adversarial fault injections designed against hypothesised failures; the instances below were found by running this system across two machines for a month, and every one of them was present while all eleven scenarios passed.

The previous version of this paper closed by noting that transport, multi-process and long-lived deployment fault injection were future work. This section is a partial and unplanned discharge of that item.

11.4.1 Ten instances of one shape

Two of the ten were introduced by the previous version of this paper rather than by the system: the overhead figure, and the benchmark command that produced it. We count them because they were the same failure—a wired artifact whose output does not mean what its label says—and excluding the two we authored ourselves would be the least defensible edit available.

A broader internal review of this codebase identified further instances beyond these ten. We report only the ten we re-derived first-hand for this paper, and note that at least one item from that wider set has since been repaired—code-graph vector search, whose extension is now loaded per connection—which is a reason to publish a verified count rather than an inherited one.

11.4.2 What they share

What the ten share is not silence. We said that in an earlier draft and the table above refutes it: one returns 503 on every route, one raises `no such column`, and one is a documented command pointing at a three-year-old interpreter. Those are loud. What they share is that **the signal, where there was one, did not name the thing that was wrong**. A 503 says the store is unavailable when the store is fine. `no such column` names a column when the fault is a

Table 9: Mechanisms that were implemented, reachable on a live call path, and ineffective or unobservable at their final connection. Each was verified directly against source and stores; the middle column names the signal an operator would have consulted.

Mechanism	What the operator saw	What was true
Channel-selection bandit	1,405 plays on this store, times-tamps advancing	posterior mean 0.5 on 459/459 arms across three stores (5,657 plays); no arm holds any preference
Source-trust model	629 observations recorded	posterior mean 0.5 on 37/37 sources; every recorded reward 0.5
Trust-modulated retention	feature configured and called per fact	guard reads a column absent from the table it tests; $\tau = 1.0$, so $\lambda_{\text{eff}} = \lambda$
Recall-health monitor	healthy	semantic retrieval off for over an hour; liveness was inferred from the probe output it gates
Availability gate on a data invariant	503 on every route, indefinitely	the store was serving correctly; one drifted row failed a re-checkable data condition that ordinary use can re-violate
Schema bootstrap vs. deferred migration	no such column	a valid older store; startup indexed a column a deferred migration adds
Editor-plugin distribution	correct version, everywhere	two versions serving one store; a private data directory made a 611 MB store present as 28 KB
Writer-unavailability reporting	one message, three causes	a stale-epoch rejection was indistinguishable from a disk stall
Governance-overhead figure	a cheap, measured control-plane cost	the quantity was never an overhead: the two arms differenced are not comparable, and two attempts to say why were both wrong
Benchmark entry point	documented command	the documented interpreter carries a release three years old

migration that has not run. Loudness and diagnosticity are independent, and only the second is useful.

It is worth separating the subclass that is genuinely silent, because it is the harder one and it is where our two checks apply. In four of the ten there is no signal at all: nothing raised, nothing logged at warning or above, every file present and every version string correct. A learner sitting at its prior mean and a decay multiplier reducing to one produce output identical to correct operation. Those four would not appear in a coverage report as uncovered, and none is a crash, a leak, or a wrong answer to a user’s question. The remaining six fail loudly, or partially, or outside the runtime altogether—two are artifacts of the previous version of this paper rather than of the system—and we flatten them at our peril. Two properties recur across the set.

Implemented, reachable and effective are three different questions. A grep answers the first. A call-graph trace answers the second. Neither answers the third. Answering it requires an oracle over the *effect* the mechanism is supposed to produce, and that oracle has to be independent of the mechanism. For a learner or a schema-guarded join, persisted state is the convenient place to find one, which is what our two checks use. It is not the general answer: process availability, package distribution and experimental validity each need a different independent observer, and three of the ten could not have been settled by querying a store at all. The principle is oracle independence. Querying the store is one instance of it.

A signal derived from the mechanism it evaluates cannot detect that mechanism’s failure. A liveness probe reading the output it gates, a play counter standing in for learning, a version string reported by whichever copy started first, a latency delta whose two arms were assumed to differ only in governance. In each case the instrument and the subject were the same component, so the failure was outside the instrument’s range.

These two properties are why the usual instrumentation does not help. Adding logging to a mechanism that believes it is working produces confident logs; adding a metric derived from its own output produces a healthy metric.

11.4.3 Threats to validity

This is not a controlled experiment and we do not present it as one.

- **One codebase, two machines, no control condition.** Whether the prevalence generalises is untested. What generalises, if anything, is the *shape*—and the two invariants of [Section 7.4](#) are the falsifiable part: point them at another system and they either find instances or they do not.
- **Found by operation, not by search.** There was no systematic sweep that would let us report a denominator. We cannot say what fraction of this system’s mechanisms are affected, only that these ten are.
- **Selection is toward the visible.** An inert mechanism nobody happened to query is by construction absent from this table. The count is a floor.
- **Two authors of the finding are also the authors of the system,** which is what makes the first-hand re-derivation and the shipped checks the substance of the claim rather than the narrative around it.
- **One unexplained failure is excluded.** A single segmentation fault was observed once during full-suite runs, in garbage collection inside the validation layer with a vector-store event loop live. It has not been root-caused and does not appear in [Table 9](#); a section about invisible failures that hid its own unexplained one would not be worth reading.

12 Limitations and Threats to Validity

Authorization-layer isolation only. Tenant and scope isolation are enforced by the authorization layer; a single OS user with direct filesystem access to the SQLite stores is outside the threat model.

Mesh validated only at small scale. Per-device revocation and key rotation, WAN-latency and network-partition fault injection, and multi-machine operation beyond a two-node loopback remain future work. Mesh is coordination—not consensus, quorum, or a CRDT database.

Temporal accuracy not benchmarked end-to-end. We measure temporal *mechanisms*; we do not report agent-task accuracy against an external benchmark. The cross-surface `as_of` demotion path is newly wired and not yet benchmarked. The `EbbinghausLangevinCoupling` class is experimental with no production caller.

Data-subject rights are workspace-scoped, not member-scoped. Erasure and export operate on a profile. A member of a shared workspace cannot exercise access or erasure over *only their own* contributions, because no member-scoped erasure path exists and the erasure operation is owner-only. The obstacle is identity, not plumbing: facts carry no author column, and the provenance table that does carry one covers 69.8% of facts with 3,997 of 4,340 authors recorded as opaque capability digests and 237 as `unknown`. For a single-operator deployment this is immaterial. For a shared workspace under GDPR it is a real limitation on Article 15

access and Article 17 erasure at member granularity, and a deployer should treat the profile, not the person, as the unit those rights attach to until a member-scoped path exists.

Verified erasure is scoped to live stores. The completeness gate covers the canonical store and the registered projection owners. Backup artifacts and any remote peer that has received state are outside it, and the completeness flag is withheld while backup obligations are pending rather than asserted over them. The audit chain establishes same-host tamper evidence; it is not a remote trust anchor and does not attest to another machine.

Learning is shown under ablation, not in deployment. [Section 7](#) reports a channel-selection bandit, a source-trust model, and a trust-modulated retention term that were wired and had never changed an outcome at the version whose field measurements this paper reports. Two had their named cause repaired afterwards, and [Table 4](#) shows under ablation that the repaired path carries a signal from an agent’s behaviour to a posterior. Three limits on that result, all of which matter. The engagement it consumes is generated by the experiment, not observed in the field, so it establishes transport and not preference quality. It says nothing about whether the arms a real deployment converges on are ones an operator would endorse, because we ran no such deployment. And in the authors’ own live store every arm was still at its prior at the last reading, which is what a single-operator installation over days looks like whether the loop works or not. The third mechanism has a scoped repair named in [Section 14](#) that was not performed. No other result in this paper depends on any of them.

The failure class is not a controlled study. The ten instances of [Section 11.4](#) come from one codebase on two machines, found by operating the system rather than by a systematic sweep, so we can report no denominator and no prevalence. An inert mechanism nobody happened to query is by construction absent, which makes the count a floor. The falsifiable part is the two invariants: pointed at another system they either find instances or they do not.

One failure is unexplained and one is unresolved. A single segmentation fault was observed once during full-suite runs, in garbage collection inside the validation layer with a vector-store event loop live; it has not been root-caused. Separately, the generation-fence scenario failed one repetition in 1,200 through a transient writer unavailability that is now diagnosable but not diagnosed.

The cost of governance is measured by component, not end to end. [Section 11.2](#) withdraws the previous version’s overhead figure because the two arms it differenced are not comparable, and [Section 11.2.1](#) replaces it by timing the envelope’s components where they run. That result is in-process and on a temporary filesystem, so the journal components in particular will differ on a persistent device, and the sum of component medians is not the median of the sum. It establishes the *distribution* of cost within the envelope—which is the part that carries the finding—more firmly than it establishes any single absolute number.

Reliability evaluation is component-level. The harness drives real production modules over fresh per-trial SQLite databases but does not exercise the HTTP/MCP/WebSocket server stack, inter-process communication, or production-scale concurrency. The `sqlite-vec` ANN index is not loadable in the harness environment; `expl`’s vector erasure is scoped to the `embedding_metadata` SQL table. Retained result artifacts record package version and platform but not a source commit or wheel hash.

External long-context benchmarks are future work. No new V4 LoCoMo run is reported; V3-architecture LoCoMo results are carried forward, protocol-scoped. An end-to-end V4 benchmark against LoCoMo and LongMemEval is preregistered.

Single-node local-first scope. All reliability results are from single-machine deployments; multi-node mesh behavior and distributed consistency are not characterized.

Concurrent and closely related prior art. MemTX and MemTxn [\[2, 11\]](#) on adjacent transaction layers; MemClaw / Governed Memory [\[15, 24\]](#) on governed multi-tenant shared memory; PROJECTMEM [\[14\]](#) on local-first event-sourced agent memory. SLM 4.0’s position is a distinct local-first composition, not priority over any of these.

Corpus-bounded novelty. “Not found in the searched corpus” is a dated review through 2026-08-03, not proof of first invention.

13 Novelty and Contributions

We state novelty with corpus-bounded language: “not found in the searched corpus” means not described in primary sources reviewed through 2026-08-03—a dated review, not a priority claim. Several capabilities SLM 4.0 provides—multi-tenant isolation, compliance controls, temporal memory, local-first operation—exist individually in shipping systems. We claim (a) their *integration* into one governed, local-first, verifiable runtime under a governing admission invariant (enforced on the primary write path; target architecture for all surfaces), and (b) specific innovations the V2–V4 line of work contributes.

C1 — A failure class, two invariants that expose it, and the repair one of them produced (lead). Ten mechanisms in this system were implemented, reachable on a live call path, and ineffective or unobservable at their final connection, every one of them silently (Section 11.4). Two properties recur: *implemented*, *reachable* and *effective* are three different questions, of which only the third requires querying the store; and a signal derived from the mechanism it evaluates cannot detect that mechanism’s failure. We contribute two mechanical checks, released in 4.1.5, that answer the third question—a prior-distance assertion over Bayesian learners, and a join-liveness assertion over schema-guarded paths that reports where a guard’s missing data actually resides (Section 7.4). The contribution does not stop at detection. The prior-distance verdict prompted an investigation that found two defects no counter in the subsystem could reach—two individually correct components disagreeing on which identifiers count, and a neutral-by-default settlement that shrank posterior variance while holding the mean—and their fixes are public in 4.1.6–4.1.9 (Section 7.6). We then test the repair under control rather than assert it. A three-arm ablation varying only the identifier namespace (Table 4) moves no posterior with the defect present, moves every instantiated arm with it absent, and—in a negative control that writes every outcome ticket but supplies no engagement—correctly settles nothing at all. We still make no attribution claim about the historical field state, because we ran no replay against it, and a controlled ablation is not a deployment result.

Not found in the searched corpus *for agent memory*, with three honest qualifications. Wu [37] already established silent failure in a production agent runtime as a studied phenomenon with a mechanism-oriented taxonomy, and we claim no priority over it; our class is the sub-case in which no error event occurs, which its five classes do not cover and which its human-observation finding could not surface. Detecting a conditional branch that never executes in production is long established in software engineering as dead-conditional and flag-controlled dead-code analysis, and we claim no novelty for the idea; our narrow claim is its application to schema-presence guards on a learning path, where the consequence is a learner that is arithmetically inert rather than merely disabled. And MemFail [33] diagnoses memory failure modes from outside the system across summarization, storage and retrieval—a different axis, on which none of our ten instances appears, because a black-box benchmark sees a plausible answer throughout. *Scope*: these are admission and observability claims. We present no evidence that repairing any of the ten improves agent behaviour, because we have run no such experiment.

C2 — Verifiable memory transactions (co-lead). An atomic canonical commit with a transactional obligation ledger, per-projection apply/verify/compensate/erase owners under generation fencing, and a hash-checkable completion manifest (COMPLETE / DEGRADED / FAILED). Distinct from and composable with concurrent belief-level [11] and logical-state [2] transactions. Manifest correctness and real `compensate()` behavior are *measured* (exp2).

C3 — Governance-native memory with deployment-context assessment (co-lead). A scoped governance control plane in a local-first runtime: immutable server-derived actor identity, pure kind-keyed policy registry, role-based access, write-time generation-fenced isolation with verified projection-store erasure, hash-chained audit, a non-authoritative EU AI Act checklist that abstains without intended-use context, and operable recovery (auto-reconciliation, a write-path stall watchdog, and an RBAC-gated dashboard/CLI/MCP remediation surface for stuck or degraded operations). Cross-tenant isolation (exp5), erasure completeness (exp1), generation-fenced admission (exp7), and policy evaluation (exp8) are *measured*. *Scope*: authorization-layer isolation, not legal certification.

C4 — SLM-Mesh: serverless governed coordination (major). Serverless, per-tenant-isolated coordination through a local SQLite broker, with optional remote peer discovery and message proxying, across all three operating modes. *Scope*: leaderless LWW, opt-in, default-profile, two-node-loopback validated; not consensus/quorum/CRDT; WAN-scale and per-device revocation are future work.

C5 — Time-aware memory (major). A shipped temporal stack: three-date model, bi-temporal storage, dedicated temporal retrieval channel, non-destructive superseded-fact demotion, Ebbinghaus recency model, and cross-surface `as_of` recall. Three temporal behaviors measured directly.

C6 — Information-geometric retrieval brain, productionized (major). Five candidate producers fused by RRF, over Fisher-Rao scoring, sheaf-cohomology consistency, and a Riemannian–Langevin lifecycle, together with an entity knowledge graph and unified caching and compression. Contribution is productionizing and governing this brain inside a multi-tenant runtime.

C7 — Direct reliability evaluation method (supporting). A bracketed, positive-controlled, fail-loud fault-injection harness measuring agent-memory durability, isolation, admission, and temporal guarantees against the candidate package.

C8 — Governed skill evolution: a design, not a result. A blind-verified, budgeted pipeline under per-cycle budget controls with append-only, hash-linked status transitions to a quarantine store. The previous version presented this as a major contribution. **We demote it.** The pipeline is implemented and gated and *has never executed*: its cycle is entered and exits having found no candidate, and the evolution log is empty. A design for bounding self-modification is worth describing, and we describe it in those terms; it is not a delivered capability, and the corpus comparison is therefore withdrawn until it runs. The withdrawal is not merely procedural: Tang et al. [38] crystallises evidence-backed policies from agent memory into callable skills carrying verification rules and applicability boundaries, and reports results on two benchmarks. That is the same territory, executing, where ours is gated and idle. *Scope*: default stops at quarantine; an explicit configuration flag—not human or RBAC approval—can activate promotion; parent-version lineage and generation tracking are not reported.

14 Future Work

Memory scaling via quantization. SLM 4.0’s context-optimization layer combines exact-match caching with reversible lossless compression to reduce working-set pressure at the session boundary. As deployment stores grow to tens or hundreds of gigabytes—a realistic trajectory for long-lived enterprise agents—a near-lossless vector-quantization tier for the semantic-channel embedding store becomes necessary. Gao and Long [4] showed that a codebook-free, random-rotation

1-bit quantizer achieves a provably tight error bound and outperforms product quantization on ANN recall at near-zero indexing cost; Zandieh et al. [29] extended this line, combining a polar-coordinate preprocessing stage [7] with a 1-bit quantized Johnson–Lindenstrauss residual corrector [28] to reach near-optimal distortion across bit-widths. The natural SLM 4.0 extension is a pluggable, data-oblivious quantization tier in the optimize layer that compresses stored embeddings without changing the retrieval interface, with the existing lossless path as a strict-reversibility fallback.

Graph engineering and knowledge-graph maturation. SLM 4.0 maintains an entity knowledge graph with spreading-activation retrieval, but graph construction currently relies on LLM-extracted entities without automatic temporal-edge invalidation or community-level organization. Graphiti [22] demonstrates one concrete realization—an episode/entity/community architecture where a contradiction sets an `expired_at` timestamp on the superseded edge rather than deleting it, and a label-propagation step groups entities into communities with maintained summaries. Integrating temporal-edge invalidation into SLM 4.0’s entity-graph channel and adding community summaries as an additional retrieval signal are well-scoped extensions that would strengthen multi-hop and cross-entity recall.

Temporal memory: point-in-time reconstruction and calibration. SLM 4.0’s `as_of` path performs superseded-fact demotion at query time but does not reconstruct full system state at an arbitrary past instant—in particular it does not yet apply `as_of` to the backup store or the sqlite-vec ANN index. Extending temporal-validity filtering to the ANN index and the restore path would close that scope gap and make point-in-time audits complete; wiring the (currently unwired) Fisher–Langevin posterior coupling would make the Ebbinghaus decay adaptive rather than fixed-parameter; and an ablation against the temporal categories of LoCoMo [13] and LongMemEval [26] would quantify the accuracy contribution of bi-temporal demotion.

End-to-end agent retrieval benchmark. The evaluation here measures structural correctness at the component level; it is not a retrieval-accuracy benchmark. A dedicated V4 suite should run SLM 4.0 as a live recall server across the LongMemEval [26] abilities and the LoCoMo [13] categories, exercise multi-session write/recall cycles that trigger the generation fence and the completion manifest, and report retrieval degradation under fault injection. He et al. [8] further show that LoCoMo-saturating agents still fail when memorization and action are coupled across sessions, motivating an additional pass on interdependent multi-session workflows.

SLM-Mesh at WAN scale and per-device revocation. Extending the mesh beyond a LAN raises three problems: consistency under arbitrary partition (a causal-consistency or explicit CRDT model to remain correct across disconnect/reconnect); per-device revocation (invalidating a decommissioned device’s state across peers without a central coordinator); and message-content scrubbing before transit (mesh `state` values pass a secrets filter, but `mesh_send` payloads are not yet scrubbed pre-transmission). Each is a scoped, engineering-grade extension with clear acceptance criteria.

Persistent generation fence. The generation fence is enforced in process memory and does not survive a restart; persisting the fence epoch to the control store and rehydrating on startup is a self-contained fix and remains open.

The watchdog item from the previous version, and what doing it taught. The previous version listed a kill-and-replace path for a stalled write-path worker, with restart backoff and alerting, as future work. It was implemented. The result is the first row of Table 9: the revival monitor reported healthy while semantic retrieval was off for over an hour, because it

inferred liveness from a probe search and discounted zero results—which is exactly what a dead embedder produces. The stated fix was correct and insufficient, and the insufficiency generalises: **a liveness check must not derive its signal from the output of the component it is checking.** Whether that rule holds beyond this instance is unproven and is the more interesting open question.

Showing that a repaired learner learns. [Section 7.6](#) removes the barrier on two of the three mechanisms and stops short of the result that matters: at the last reading every arm was still at its prior, because the outcomes that did arrive were themselves neutral. The open question is not whether the signal now reaches the learner—the ledger shows it can—but whether a reward derived from observed engagement moves a posterior in a direction an operator would endorse. That needs a multi-operator deployment over weeks and a held-out comparison against uniform selection, neither of which a single-operator store supplies. The third mechanism is untouched: re-keying the trust join onto the provenance table makes trust-modulated retention live with no migration, and the join-liveness check already names the table and the row count. Binding the loop ledger into the completion manifest would make execution and write verification a single chain rather than two.

Isolating the cost of governance — done differently than planned. The previous version of this paper named a build-time switch—one pipeline measured with the envelope enabled and disabled—as the way to isolate this cost, and we no longer think that is the right instrument. A switch is still a subtraction, and it carries a worse problem: a governance control plane with a documented off position is a governance control plane with a bypass, and shipping one to measure it would trade a security property for a number. [Section 11.2.1](#) takes the other route and times the components in place, which needs no switch and leaves nothing to disable.

What remains open is the part measurement cannot settle. The envelope is dominated by journal durability, so the interesting question is now an engineering one: how much of that 7.7 ms survives group commit, batched journal writes, or a journal on a separate device, and at what recovery cost. That is a design study we have not run.

Generalising the two invariants. The prior-distance and join-liveness checks of [Section 7.4](#) are implemented for this system’s learners and guards, and the implementations are specific to it in ways the pattern is not. Prior distance names two tables, assumes a $\text{Beta}(1, 1)$ prior and tests a mean; it cannot inspect a contextual bandit, a categorical posterior, a gradient learner or a learned ranker without being extended. Join liveness registers one guard by table and column name and assumes a SQLite store. What generalises is the question each asks, not the code that asks it. We have general patterns and codebase-specific probes, and calling the second general would be the kind of claim this paper exists to object to. A useful test of the claim in [Section 11.4](#) would be to run both against other agent-memory implementations and report what they find, including nothing.

Machine-checked formal proofs. The Verified Design Invariants of [Section 8](#) are established here by implementation trace and fault-injection measurement. Upgrading them to machine-checked proofs (e.g. in Lean 4 or TLA^+) would lift the guarantee from *empirically upheld at $N=200$ per trial* to *proven for all executions conforming to the protocol*. The completion manifest’s COMPLETE/DEGRADED/FAILED derivation is a natural first target: a small, bounded state space tractable for model checking.

15 Conclusion

Agent memory is becoming shared infrastructure, judged by properties that retrieval-accuracy benchmarks do not capture: isolation between tenants, verifiable correctness of canonical operations, governed self-improvement, coordination without a central server, first-class time, and operable governance. SLM 4.0 addresses a specific, under-served point in this space—a local-first runtime that co-locates multi-scope isolation, verifiable memory transactions, and governance controls under one admission path, without requiring a cloud service. The contribution is not any single mechanism—bi-temporal storage, knowledge-graph retrieval, and GDPR-aware erasure each have strong prior art—but their composition into one verifiable, governed, open-source runtime, together with a physical-store transaction spine and a measured reliability evaluation.

The fault-injection result—2,199 of 2,200 repetitions—is deliberately narrow: it shows the spine services upheld eleven deterministic structural properties at the component level; it does not claim retrieval-accuracy superiority over commercial services, nor does it substitute for an end-to-end multi-agent benchmark. Within that honest scope, SLM 4.0 delivers what its architecture promises—tenants isolated by profile-scoped predicates on every retrieval channel, with a write-time generation fence rejecting stale-epoch writes on top, every write through that path either verifiably complete across its registered projection owners or recorded as a hash-checkable DEGRADED manifest, and erasure proved complete for the three canonical projection stores—as one instance of **AI Reliability Engineering** for the memory layer. The open threads of [Section 14](#)—near-lossless quantization as stores scale, graph engineering, an end-to-end V4 retrieval benchmark, WAN-scale mesh, a persistent fence, and machine-checked proofs—are each bounded extensions. SLM 4.0 lays the control plane; the reliability and quality properties reported here are a lower bound on what a production deployment requires.

This version also retracts. A learning layer the previous version called governed had never learned, and we can now show that exactly rather than argue it approximately. A per-write governance cost we quoted differenced two arms that are not comparable. We withdraw the figure, and we withdraw both of the explanations we later gave for it. A skill-evolution pipeline we called a major contribution has never executed and is demoted to a design. The reliability harness that returned 2,200 of 2,200 in the previous version returns 2,199 of 2,200 here, and we report the one failure rather than re-running until it disappears.

We think that is the more useful contribution. Ten mechanisms in this system were built correctly, reached on a live call path, and left ineffective at their final connection, every one of them silently—and the reason they were invisible is structural rather than careless: *implemented*, *reachable* and *effective* are three different questions, and a signal derived from the mechanism it evaluates cannot detect that mechanism’s failure. The two invariants this paper contributes answer the third question mechanically. The first thing they did was find three of the ten in our own store; the second was to name a cause that no counter in the subsystem could reach, and that named cause is repaired in a public release a reader can install and inspect ([Section 7.6](#)). That is the loop we want to argue for—detect, name, repair, and state plainly what remains unproven—because holding a memory system to an evidence standard is only meaningful if the standard is allowed to take claims away.

Availability and Full Report

SuperLocalMemory is open source under AGPL-3.0 at <https://github.com/qualixar/superlocalmemory> and is packaged for PyPI and npm as `superlocalmemory`. It installs and runs on a single machine in three modes—fully local (no provider in the memory path), local with an on-device model, and provider-assisted—with no Docker, external graph database, or API key required for the default local mode. The reliability evaluation harness of [Section 11](#) and its per-experiment result records are released alongside this paper. Reproducibility is *per-result*:

the harness re-runs the fault-injection experiments (this version’s results were produced on Python 3.14.5, macOS-26.4.1-arm64, package 4.1.3), whereas the carried-forward performance and V3 LoCoMo figures depend on artifacts (a $\sim$ 1 GB store and external model calls) that are not fully hash-pinned—retained records note package version and platform but not a source commit or wheel hash. The exact audited build is pinned at release.

This preprint is self-contained. A longer internal technical report—with extended assumptions, full proofs, per-layer implementation detail, and an exhaustive capability inventory—underlies this work; it is not part of this preprint’s artifact.

Author Biography

Varun Pratap Bhardwaj is a Solution Architect and the founder of Qualixar. His work centres on **AI Reliability Engineering**—the systematic application of reliability, correctness, and governance to AI-augmented software. SuperLocalMemory is his line of work on local-first, governed memory for AI agents, consolidated across the V2–V4 releases into the system described here. ORCID: 0009-0002-8726-4289.

Availability and Licensing

SLM 4.0 is released as open source under the **GNU Affero General Public License v3.0 (AGPL-3.0-or-later)**, with a separate commercial licence available for closed-source, proprietary, or hosted deployment. Source, packaging metadata (`pyproject.toml`, `package.json`) and the `LICENSE` file all declare AGPL-3.0-or-later; earlier preprints in this series that stated a permissive licence are superseded by this statement.

The reliability evidence in [Section 11](#) is not merely reported: the full experiment harness ships with the source under `benchmark/`, and the bundle in this paper is regenerated by a single runner, `benchmark/run_all.py`. A repository test asserts that every experiment on disk is reachable from that runner, so a result cited here cannot become unrunnable without the test suite failing. Readers are invited to regenerate the numbers rather than take them on trust.

Author Contributions

Varun Pratap Bhardwaj conceived the system, designed and implemented SuperLocalMemory 4.0, specified the reliability guarantees and their formal statements, designed and executed the evaluation, and wrote the manuscript.

Garima Singh contributed to the product conception and requirements definition, shaped the evaluation criteria and acceptance conditions from a product-ownership perspective, and carried out sustained independent validation over several months on separate hardware. The resulting defect reports, usability findings, and prioritisation feedback materially changed the system’s scope, its interface design, and which guarantees were treated as load-bearing.

Arun Pratap Bhardwaj established and ran the iterative validation feedback loop across the three operating modes and successive releases, contributed an operational-technology and industrial-systems perspective on reliability and failure handling, and reported the behavioural findings that drove several correctness fixes.

All authors reviewed the manuscript, take responsibility for the work as presented, and approved the submitted version.

References

- [1] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory, 2025. arXiv:2504.19413.
- [2] H. Cui et al. MemTxn: A Transaction Boundary for Source-Supported Updates and Complete-State Recovery in Agent Memory, 2026. arXiv:2607.27834.
- [3] J. Fang et al. LightMem: Lightweight and Efficient Memory-Augmented Generation, 2026. ICLR 2026. arXiv:2510.18866.
- [4] Jianyang Gao and Cheng Long. RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search. In *Proc. ACM SIGMOD International Conference on Management of Data*, 2024.
- [5] B. J. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models, 2024. NeurIPS 2024. arXiv:2405.14831.
- [6] B. J. Gutiérrez, Y. Shu, W. Qi, S. Zhou, and Y. Su. From RAG to Memory: Non-Parametric Continual Learning for Large Language Models (HippoRAG 2), 2025. ICML 2025. arXiv:2502.14802.
- [7] Insu Han, Praneeth Kacham, Amin Karbasi, Vahab Mirrokni, and Amir Zandieh. PolarQuant: Quantizing KV Caches with Polar Transformation, 2025.
- [8] Zexue He et al. MemoryArena: Benchmarking Agent Memory in Interdependent Multi-Session Agentic Tasks, 2026.
- [9] J. Kang, M. Ji, Z. Zhao, and T. Bai. Memory OS of AI Agent, 2025. arXiv:2506.06326.
- [10] Letta AI. Letta. <https://github.com/letta-ai/letta>.
- [11] X. Li et al. MemTX: Transactional Belief Commit for Stateful Agent Memory, 2026. arXiv:2607.23929.
- [12] Z. Li et al. MemOS: A Memory OS for AI System, 2025. arXiv:2507.03724.
- [13] A. Maharana et al. Evaluating Very Long-Term Conversational Memory of LLM Agents, 2024. arXiv:2402.17753.
- [14] R. C. Malo and T. Qiu. PROJECTMEM: A Local-First, Event-Sourced Memory and Judgment Layer for AI Coding Agents, 2026. arXiv:2606.12329.
- [15] Y. Margalit, N. Cohen-Inger, E. Avram, R. Taig, and O. Margalit. Governed Shared Memory for Multi-Agent LLM Systems (the MemClaw service and the ArgusFleet governance harness), 2026. arXiv:2606.24535.
- [16] Mem0. Introducing the token-efficient memory algorithm. <https://mem0.ai/blog/mem0-the-token-efficient-memory-algorithm>, 2026. Vendor technical report describing single-pass extraction, entity linking, multi-signal retrieval, and vendor-reported April 2026 benchmark results; accessed 2026-08-05.
- [17] Mem0 AI. Mem0 (incl. the OpenMemory local MCP server). <https://github.com/mem0ai/mem0>.
- [18] C. Packer et al. MemGPT: Towards LLMs as Operating Systems, 2023. arXiv:2310.08560.

- [19] V. Pratap. SuperLocalMemory: Trust and Behavioral Foundations for Privacy-Preserving Agent Memory (v2), 2026. arXiv:2603.02240.
- [20] V. Pratap. SuperLocalMemory V3.3: The Living Brain, 2026. arXiv:2604.04514. Harvard ADS: 2026arXiv260404514P.
- [21] V. Pratap. SuperLocalMemory (v3), 2026. arXiv:2603.14588.
- [22] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef. Zep: A Temporal Knowledge Graph Architecture for Agent Memory, 2025. arXiv:2501.13956.
- [23] Supermemory. Supermemory — memory API for the AI era (container-tag multi-tenancy; SOC 2 Type 2 / HIPAA / GDPR; Docker self-host). <https://supermemory.ai>, 2026.
- [24] H. Taheri. Governed Memory: A Production Architecture for Multi-Agent Workflows, 2026. arXiv:2603.17787.
- [25] Y. Wang and X. Chen. MIRIX: Multi-Agent Memory System for LLM-Based Agents, 2025. arXiv:2507.07957.
- [26] D. Wu et al. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory, 2025. ICLR 2025. arXiv:2410.10813.
- [27] N. Yang et al. SF-AMS: Strategic Forgetting for Structured Memory in LLM Agents, 2026. arXiv:2607.22562.
- [28] Amir Zandieh, Majid Daliri, and Insu Han. QJL: 1-Bit Quantized JL Transform for KV Cache Quantization with Zero Overhead, 2024.
- [29] Amir Zandieh, Majid Daliri, Majid Hadian, and Vahab Mirrokni. TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate, 2025.
- [30] Zep AI. Graphiti. <https://github.com/getzep/graphiti>.
- [31] J. Zhang et al. Beyond Similarity: Trustworthy Memory Search for Personal AI Agents (MemGate), 2026. arXiv:2606.06054.
- [32] C. Hu, X. Gao, Z. Zhou, D. Xu, Y. Bai, X. Li, H. Zhang, T. Li, C. Zhang, L. Bing, and Y. Deng. EverMemOS: A Self-Organizing Memory Operating System for Structured Long-Horizon Reasoning, 2026. arXiv:2601.02163. ACL 2026.
- [33] I. Garg, N. Kolhe, D. Song, and X. Zhao. MemFail: Stress-Testing Failure Modes of LLM Memory Systems, 2026. arXiv:2605.26667.
- [34] C. Lam, J. Li, L. Zhang, and K. Zhao. Governing Evolving Memory in LLM Agents: Risks, Mechanisms, and the Stability and Safety Governed Memory (SSGM) Framework, 2026. arXiv:2603.11768.
- [35] MindMemOS. MindMemOS: A Portable and Self-Evolving Memory Operating Layer for AI Agents, 2026. arXiv:2608.12428.
- [36] AgentMemBench. AgentMemBench: A Systematic Benchmark for Evaluating Long-Term Memory Management Strategies in Conversational AI Agents, 2026. arXiv:2608.00009.
- [37] W. Wu. When Errors Become Narratives: A Longitudinal Taxonomy of Silent Failures in a Production LLM Agent Runtime, 2026. arXiv:2606.14589.

- [38] B. Tang, Y. Zhang, G. Zhuang, W. Wei, G. Zheng, L. Xie, Y. Tan, F. Xiong, Q. Yang, E. Chung, and Z. Li. From Memory to Skills: Evidence-Grounded Co-Evolution Governance for Long-Horizon LLM Agents (MSCE), 2026. arXiv:2607.16621.
- [39] X. Mao, J. Zhao, G. Penn, and C. Wang. What Happens Inside Agent Memory? Circuit Analysis from Emergence to Diagnosis, 2026. arXiv:2605.03354.
- [40] J. H. Saltzer, D. P. Reed, and D. D. Clark. End-to-End Arguments in System Design. *ACM Transactions on Computer Systems*, 2(4):277–288, 1984. doi: 10.1145/357401.357402.
- [41] P. Huang, C. Guo, L. Zhou, J. R. Lorch, Y. Dang, M. Chintalapati, and R. Yao. Gray Failure: The Achilles’ Heel of Cloud-Scale Systems. In *Proceedings of the 16th Workshop on Hot Topics in Operating Systems (HotOS)*, 2017. doi: 10.1145/3102980.3103005.
- [42] P. Huang, C. Guo, J. R. Lorch, L. Zhou, and Y. Dang. Capturing and Enhancing In Situ System Observability for Failure Detection. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018.
- [43] C. Lou, Y. Jing, and P. Huang. Demystifying and Checking Silent Semantic Violations in Large Distributed Systems. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.
- [44] B. Souza, C. Lou, S. Nath, and M. Pradel. FlyCatcher: Neural Inference of Runtime Checkers from Tests, 2026. arXiv:2604.22028.